

General Design for Private L2

Abstract tbd

Introduction

We construct partially embedded L2s to provide a private, permissioned, customizable execution environment that stays atomically composable with everything else. The main design goal is to allow institutions to use a public blockchain for their operations, bridging the gap between private and public chains. This requires that smart contracts can be executed confidentially and that the flow of information can be controlled, while still being able to interact with public smart contracts and with those operated by other institutions or groups of institutions.

Partially embedded means that a subset of the L1 validators, or an independent set of validators, executes the L2 smart contracts, while sequencing and cross-chain messaging happen on the L1, providing increased security and trust as well as fast and reliable cross-chain messaging and transfers.

For the first version (MVP), the partially embedded L2 offers the following functionality. Their implementation is described later in the document.

- An L2 is created by sending a creation transaction to the precompile that specifies the configuration of the L2. The configuration includes the validators operating the L2. The design is agnostic to whether or not they are also L1 validators. The configuration may be changed later by a designated party.
- Smart contracts can be installed on the L2. Smart contracts can be addressed globally, i.e., they can receive messages from users and from smart contracts running on the L1 or on other L2s. Installation of smart contracts is gated by an allowlist specified at creation.
- L2 smart contracts are executed only by the L2 validators. The configuration of the L2 defines how the L1 verifies the L2's state root (a threshold of validators submitting the same state root, a state root proved by ZK, attested by a TEE, etc.). The MVP will support only validation by a threshold of validators attesting to the same root.
- The state of L2 smart contracts is private. In the first version, however, the L2 validators are privy to the state. (We will consider protecting the state, or parts of it, using TEEs, MPC, or FHE later.)
- State can be accessed by calling the smart contracts via the `eth_call` interface. Unlike the usual behaviour, such calls are gated by access control. To this end, signatures are verified and the sender's identity is provided to the smart contract, so that the contract can implement access control and determine what information different parties may obtain.

- An L2 instance has an EOA account address on the L1. This account is used to pay the gas fees for the transactions that the L2 submits (the L2 validators jointly sign them).
- Transactions to the L2 are submitted and paid for on the L1 (details of gas accounting are somewhat involved). This includes all L1 fees (including those for messages submitted by the L2 back to the L1 as part of processing a transaction on the L2) and fees on the L2.
- All tokens are native on the L1, which is the source of truth for tokens. The L2 configuration specifies the list of L1 tokens that are natively "wrapped" into the L2. (To create a new token, one first creates the token on the L1 and then changes the configuration of the L2 to include the new token. Of course, one can install any token contract on the L2, but such tokens will not be natively wrapped.)
- The design supports "atomic" transactions between an L2 and other chains (the L1 and other L2s). That is, if a transaction fails, token transfers are rolled back and all involved smart contracts are notified.
- The L2 has a public key under which transactions to the L2 can and should be encrypted. For the MVP, the public key is specified in the configuration and the corresponding secret key is given to the L2 validators as input. Later versions will allow the public key to be rotated when a validator is removed from the L2 and to be updated by the L2 validators.

There are a number of features that will make L2s easier to use. However, they are not part of the MVP. These include the following (in addition to what has already been mentioned as out of scope above):

- Gas sponsorship. In the MVP, gas for all (external) L2 transactions is paid on the L1 when they are submitted there. This requires submitters to have an account on the L1. In the future, we want to allow an L2 to sponsor gas payments. The design of a (general) gas sponsorship mechanism is out of scope for the MVP.
- It is often difficult for institutions to hold volatile cryptocurrencies. For end users, it is cumbersome to obtain and hold cryptocurrency for gas payments. It is therefore desirable to support fee payment in a stablecoin. The MVP will only support fee payment in \$MON.

Impact of state being private

Many Ethereum mechanisms and L2 designs require that the state be public. Examples are transaction validation when a block is formed (checking whether the sender's balance covers the transaction value), accessing state information via `eth_call`, and the estimation of gas usage (which can no longer be simulated when the smart contracts and their state are not public). We therefore have to provide alternative solutions for such mechanisms.

The state being private and not accessible to all L1 validators also means that interactions between different L2s cannot be synchronous. This requires more elaborate transaction and context handling in smart contracts and makes gas estimation even harder.

Security models and assumption (TBD)

The security of the L2 is rooted in the security of the L1. L2 leverages L1 censorship resistance and robust message routing across the ecosystem. An L2 participant typically trusts the L1 and their specific L2, though trust may not extend to other L2s. While these guarantees differ from native L1 security, the model is highly effective when users and institutions trust the designated L2 validators. This asymmetric security framework is centered on established trust relationships, although integrating TEEs or ZK-proofs would provide significantly stronger verification.
(needs to be finished).

Use case example scenarios

We consider two exemplary use cases that are based on existing pilots and prototypes.

Wholesale cross-boarder CBDC

This example considers the settlement of high-value payments between financial institutions across currency and jurisdictional boundaries. It identifies the participants and their objectives, the deficiencies of present arrangements, the technical and legal requirements that any acceptable solution must satisfy, and the extent to which prior central bank experiments have satisfied them.

Wholesale settlement and its participants

Wholesale settlement is the movement of value between financial institutions, rather than between an institution and the public. Volumes are low and individual values are high, and participation is restricted to institutions that hold accounts at a central bank or are otherwise made eligible by it. Wholesale central bank digital currency is accordingly not a new form of money but a tokenised representation of reserve balances that already exist, the same liability of the same issuer, recorded on a programmable ledger. It is distinct from retail central bank digital currency, which is a claim held directly by the public, and from stablecoins and tokenised deposits, which are liabilities of private issuers. A platform serving this use case is not open to retail participants.

There are five classes of participants. *Central banks* issue and redeem tokenised reserves in their own currency and set the terms on which those reserves may be held; they require that monetary sovereignty, policy transmission and financial stability remain unaffected, that access eligibility and holding constraints remain theirs alone to determine, and that they retain at least the supervisory visibility they have today. They do not seek involvement in the approval of individual payments, and they will not accept liability for infrastructure they do not govern. *Commercial banks* act as debtor agent, creditor agent or intermediary, and in a cross-currency

payment one of them additionally supplies the conversion; they seek reduction of settlement and credit exposure, release of liquidity currently immobilised in pre-funded correspondent accounts, and fewer reconciliation breaks and investigations, while requiring that customer relationships, correspondent arrangements, pricing inputs and compliance decisioning remain theirs and remain undisclosed to competitors and to the platform. *Corporates and other end clients* are indirect participants who never transact on the platform; their requirements are predictability of timing, certainty of the amount received, transparency of fees, and visibility of status. *Supervisors* require auditability and continued effectiveness of financial-crime controls. *The operator* of any shared component is a typical fifth participant whose remit, governance and liability no existing project has settled.

The following example illustrates how these roles combine. An enterprise in jurisdiction A instructs its bank to pay 60 units of currency BBB to an enterprise in jurisdiction B. At a BBB/AAA rate of 1.2, the debtor is charged 72 AAA. Bank A1 holds no correspondent relationship with the creditor's bank B1, but does hold one with A2, which in turn holds one with B1; A2 therefore acts as intermediary and supplies the currency conversion. The resulting balance movements are:

1. The debtor's account at A1 is debited 72 AAA within A1's own systems. Neither enterprise appears on the platform at any point.
2. 72 AAA moves from A1 to A2 in tokenised reserves on jurisdiction A's ledger, under central bank A's authority.
3. A2 absorbs the conversion on its own balance sheet, having determined the rate internally and disclosed only the resulting pair of amounts.
4. 60 BBB moves from A2 to B1 in tokenised commercial bank deposits.
5. B1 credits the creditor 60 BBB within its own systems.

The two interbank legs are conditioned on one another and settle together or not at all; the two customer legs are ordinary account entries outside the platform. Central bank A governs the first interbank leg. Central bank B is not a party to the second, which settles on commercial balance sheets, whether a given leg settles in reserves or in deposits is a commercial decision between the two institutions concerned, and both routes are available for either leg.

The problem

Wholesale cross-border payments are settled today through chains of correspondent banks. Where the sending institution holds no direct relationship with the receiving institution, value moves through one or more intermediaries, and each hop settles on a commercial bank's balance sheet rather than in central bank money. Every leg therefore carries credit and liquidity exposure to the next participant in the chain. Processing is sequential: each institution can act only once the preceding one has completed, so compliance checks are duplicated, data quality degrades along the chain, and discrepancies typically surface only after settlement, when they are most costly to resolve. Each bilateral relationship must be pre-funded in the destination currency, immobilising liquidity across large numbers of accounts. For cross-currency payments

the position is worse still: outside the eighteen currencies covered by CLS, the two legs of an FX settlement are not conditioned on one another, leaving participants exposed to loss of principal if a counterparty fails between deliveries.

What tokenised central bank money offers

Tokenised central bank reserves address this by extending across borders a guarantee that already holds within them, that interbank settlement occurs in the risk-free asset. Recording reserves and commercial bank deposits on a programmable ledger allows the balance updates that constitute a payment to be conditioned on each other, so that either all of them occur or none do. This eliminates principal and settlement risk in end-to-end payments, and makes payment-versus-payment settlement available for any currency pair rather than only those served by existing arrangements. It further allows the validations that currently run serially at each hop (sanctions screening, AML and fraud controls, beneficiary confirmation, path agreement) to be performed in parallel and their outcomes exchanged before any liquidity is committed, and it decouples execution from the operating hours of domestic settlement systems.

Architectural constraints

The use case does not, however, admit a single shared ledger. Central banks require that their reserves remain under domestic authority: issuance and redemption, access eligibility, holding limits and operational parameters must be governed in the jurisdiction that issues the currency, and no external operator may hold a veto over, or visibility into, the state of a domestic reserve ledger. Commercial participants require equivalent confidentiality from one another: customer data, correspondent relationships, pricing inputs and compliance decisioning must not be disclosed to competitors or to the platform itself. Settlement must nonetheless be atomic across jurisdictions, and finality must be deterministic and identifiable as a single legal event. These four requirements, per-jurisdiction authority, mutual confidentiality, cross-partition atomicity, and unambiguous finality, are what motivate the partitioned private execution model described in the remainder of this document.

Legal requirements

Legal analysis in this domain reduces consistently to six requirements, which constrain the architecture as firmly as the technical ones.

Legal categorisation of tokenised money. The status of a tokenised reserve must be determinable in each participating jurisdiction: currency, a claim on funds held on account with the central bank, or a transferable receipt evidencing such funds. Tokenisation must not alter the legal nature of the underlying money, nor the balance-sheet relationship between issuer and holder. Where local law does not recognise currency in digital form, either statutory amendment or a receipt-based construction is required.

Settlement finality. There must be a single, identifiable event at which a payment is deemed made, complete, irrevocable and final, enforceable in each relevant jurisdiction and protected against insolvency clawback. Technical irreversibility of ledger state is a necessary but insufficient condition: the settlement trigger must be defined in an applicable rulebook and recognised in law. Multi-leg transactions must produce one unambiguous, auditable outcome rather than finality per leg.

Capacity and access. Each central bank's statutory powers must permit its own participation and permit the intended class of holders, including any foreign institutions. Eligibility to hold or transact in a given currency must be determinable and enforceable on-platform rather than only contractually.

Financial crime obligations. AML, CFT, sanctions and fraud obligations remain with each regulated participant and are not assumed by the platform. Confidentiality and selective-disclosure mechanisms must not impair record-keeping, transaction monitoring, or the ability of authorised parties to produce audit evidence on request.

Data protection and localisation. Payment data may contain personal data subject to divergent regimes and to restrictions on cross-border transfer, and ledger records cannot practicably be erased. Personal data should therefore not be recorded on a shared ledger in clear, encrypted or hashed form; identifying data should remain in the controlling institution's own systems and jurisdiction, with only minimal non-identifying commitments anchored on the shared layer.

Contractual architecture. Statute will not cover the arrangement in full. A participation agreement, platform operating terms defining the settlement event, and per-currency terms of use governing foreign holders are required, together with allocation of liability, business continuity obligations, and dispute resolution procedures.

Prior realisations in BIS Innovation Hub projects

Project mBridge (2022) implemented the use case as a single shared permissioned ledger on which four central banks issued their currencies and both domestic and foreign commercial banks held them directly. Central banks operated the validating nodes under a HotStuff+ consensus and onboarded their own commercial banks. Jurisdictional control was retrofitted onto that shared environment through token-level parameters: only domestic banks could issue or redeem against reserve balances, eligibility lists and holding limits constrained offshore circulation, and each central bank could specify how long its currency was permitted to exist on the platform — one participant restricted its currency to intraday existence only. Confidentiality rested on pseudo-anonymous, self-generated key pairs, limiting visibility of transaction detail to the counterparties, their respective central banks and the currency issuer. The pilot settled real value across four currencies, but the platform ran in a single centralised cloud environment, which the report identified as a privacy concern, and most policy controls were applied manually.

Project Mariana (2023) introduced partitioning. Three domestic platforms on permissioned Ethereum held the participating currencies, while a separate transnational network hosted an

automated market maker that priced and settled spot FX against a pooled three-currency reserve. Central-bank-operated bridges, each secured by a relayer quorum, moved currency between the layers. The project concluded that central banks can control their currency without owning the platform on which it circulates, but found bridge construction to be the dominant source of complexity even between two variants of the same protocol, and placed privacy entirely out of scope. Its principal quantitative result — that a single 50 million trade at one basis point of cost required a pool of roughly 1.8 billion, established the liquidity cost of collapsing trading and settlement into a single pre-funded step.

Project Agorá (2026) carried the partition to its conclusion. Tokenised reserves sit on jurisdictional ledgers, one per currency area, each operated under the authority of the issuing central bank; tokenised commercial bank deposits and workflow state sit on a shared unifying ledger. A coordinator contract on that unifying ledger sequences a five-stage payment lifecycle but never triggers action on a jurisdictional ledger directly, participants submit their own transactions locally, and the coordinator advances only on confirmed outcomes. Confidentiality is no longer pseudonymity on a readable ledger but cryptographic commitment at the token layer combined with participant-scoped execution off the shared ledgers, so that neither balances nor institutional logic are exposed. Atomicity is obtained by reserving balances, delegating authority to per-leg contracts, and issuing a single commit-or-cancel verdict.

Taken together, the three establish the requirements and locate the remaining gap. mBridge demonstrated that jurisdictional control can be expressed as token-level policy, but not that it survives a shared execution environment. Mariana demonstrated that partitioning preserves control while making the interconnect the hardest component to build and secure. Agorá demonstrated that partitioning and strong confidentiality are compatible, at the cost of reconstructing atomicity as application-level choreography across ledgers that share no common ordering authority. Its report is explicit that "*atomic settlement* refers to a workflow-level outcome across payment legs, not a claim about finality or synchronous multi-ledger transaction execution." Supplying that common ordering authority, without surrendering per-jurisdiction control or inter-participant confidentiality, is the problem this architecture addresses.

Cross-border worker payouts and dollar balances

This section states a second use case the architecture is intended to serve: the payment of individuals in one jurisdiction by businesses in another, where the recipient's requirement is not only to receive value but to retain it in a currency other than their own. It identifies the participants and their objectives, the deficiencies of present arrangements, the technical and legal requirements any acceptable solution must satisfy, and how one production deployment (Deel on Tempo) satisfies them.

The use case and its participants

Where the wholesale use case concerns settlement between financial institutions, this one concerns the last mile to an individual. A business engages contractors in multiple countries and

must pay each of them in a form they can hold, access and spend locally. The payment itself is comparatively small and the volume is high — the inverse of the wholesale profile — and the recipient is not a supervised entity but a consumer. The value at stake is not the elimination of credit exposure between banks but the preservation of purchasing power at the point of receipt. Seven classes of participant appear. Client businesses engage and pay workers across borders; they require compliant engagement, a single predictable cost, and no obligation to establish local banking or entity presence in each country. Recipients are individual contractors; they require that the value they were promised arrives intact, that it remains denominated in the currency it was paid in, that it is accessible without a separate application or custody burden, and that it can be spent. The platform intermediates the relationship, performs identity verification, orchestrates payout, and bears the regulatory perimeter; it requires auditable visibility over balances and flows, and confidentiality of those balances from the public. The token issuer issues the payment instrument and holds the corresponding reserves, and is separately regulated as such. The wallet provider provisions and secures custody on the recipient's behalf. A yield venue, where an earn feature is offered, generates return on idle balances and is deliberately distinct from the issuer. The settlement network carries the transfers and supplies the account, fee and confidentiality primitives on which the rest depends. The following example illustrates how these roles combine.

1. A business in the United States engages a contractor in Argentina at 2,000 USD per month.
2. The business funds payroll in a stablecoin held on the settlement network. The transfer to the platform is a single instruction covering many recipients.
3. The contractor, having completed identity verification with the platform, is provisioned an embedded wallet without separate onboarding.
4. 2,000 units of a USD-denominated token are credited to that wallet. The contractor holds dollar value directly; no conversion to local currency occurs at any point.
5. The contractor optionally routes the idle balance into a yield venue and may withdraw from it at any time.
6. The contractor draws on the balance by card, by redemption to local currency at a moment of their choosing, or by transfer to an external wallet.
7. The distinguishing property is step 3. In the arrangement this replaces, the same 2,000 USD would have been converted to pesos on arrival, at a rate and time the contractor did not choose.

The problem

Cross-border payment of individuals is executed today through correspondent banking, local payout partners, or remittance networks, and terminates in a local bank account denominated in local currency. For a recipient in a stable-currency jurisdiction this is unobjectionable. For a recipient in a jurisdiction with high inflation or currency restrictions it destroys a material part of the payment: the worker is paid in a hard currency, and that currency is converted at the last mile, at a rate they do not control, at a moment they did not choose.

Holding the original currency instead is generally impracticable rather than prohibited. Local banks may not offer foreign-currency accounts to individuals, may impose documentation

requirements and minimum balances that a contractor cannot meet, and exchange controls may cap the amount of foreign currency a resident may acquire. The result is that the recipient's exposure to their own currency is involuntary. Secondary deficiencies compound it: payout arrival is slow and unpredictable, fees are deducted opaquely along the chain, and the recipient must reconcile a payment they cannot track.

What tokenised dollar balances offer

Recording a fully reserved, redeemable claim on a hard currency as a transferable token, and delivering it directly to the recipient, removes the forced conversion. The recipient holds dollar value in a wallet rather than dollar value in transit to a peso account, and elects if and when to convert. Because the token is transferable on a shared network, the same balance can be spent by card, redeemed through the platform, moved to an external wallet, or placed in an interest-bearing venue, without the recipient holding a bank relationship in the currency of denomination.

Two secondary properties follow from the settlement network rather than the token. Delivery is immediate and its cost is fixed and knowable in advance, so the amount the recipient sees is the amount the business sent. And because balances are addressable programmatically, the platform can operate the entire loop (funding, payout, holding, yield, spend) on one ledger, eliminating the reconciliation and chain-hopping that a multi-provider arrangement imposes.

Architectural constraints

The economics of this use case are the inverse of the wholesale case and generate different constraints. There are very many accounts holding very small balances, so any per-account storage cost, any minimum balance, and any requirement that the holder acquire a separate asset to pay fees are each individually disqualifying. Transaction cost must be fixed and denominated in the unit being transferred, not in a volatile network asset, because it is charged against a consumer payment and must be absorbed predictably by the platform.

The recipient is not a cryptographic operator. Custody must be provisioned automatically on the basis of an identity check already performed, authenticated by a device credential, and recoverable; exposure of a private key or a recovery phrase to the recipient is a product failure, not a security feature. Confidentiality is required from the public, not between participants: an individual's earnings history and balance must not be publicly inspectable, while the platform must retain full visibility for audit, support and supervisory purposes. This is the opposite confidentiality posture to the wholesale case, and it is satisfied by a single trusted operator rather than by mutual concealment.

Finally, eligibility must be enforceable at the token or venue layer rather than only at the application layer, because a transferable balance can be moved by its holder to destinations the platform does not control.

Legal requirements

The requirements below continue the numbering of the wholesale use case. They differ from it substantially, principally because the recipient is a consumer and the issuer is private.

Issuance and reserve regulation. The payment instrument is a regulated payment stablecoin or electronic money token. Its issuer must be authorised as such, must hold reserves of the required quality on a one-to-one basis, must redeem at par on demand, and must attest to reserves at the prescribed interval. Materially, the principal regimes prohibit the issuer from paying interest or yield on the instrument itself, which forces any return to be generated outside the issuance.

Transmission and local payout permissions. Delivering value into each destination jurisdiction requires money transmission or equivalent authorisation, an available exemption, or a licensed local partner. The platform, not the settlement network, holds this perimeter.

Characterisation of yield. An optional return on idle balances risks characterisation as a security, a collective investment scheme, or deposit-taking, depending on jurisdiction. Structural mitigations — explicit opt-in, no promised rate, no lock-up or minimum, and return generated by a third-party venue rather than promised by the platform or issuer — must be present by design rather than asserted in disclosure.

Identity, sanctions and transfer-information obligations. Customer due diligence is performed on the individual recipient and sanctions screening applied in jurisdictions of elevated risk. Where a recipient transfers value onward to an address outside the platform's control, transfer-information obligations for payments to unhosted wallets apply and must be satisfiable without the platform having visibility of the destination.

Exchange controls and currency regulation. Delivering foreign-currency value to a resident of a jurisdiction that restricts residents' acquisition of foreign currency may be lawful, restricted or prohibited, and the analysis is jurisdiction-specific. This is the requirement most likely to constrain geographic availability and must be assessed per corridor rather than per region.

Consumer protection, custody and disclosure. The recipient must be informed that the balance is a claim on a private issuer and not a bank deposit, that it is not covered by deposit protection, and how redemption operates. Where custody is self-custodial, credential loss and recovery must be addressed as a consumer-protection matter.

Tax and employment characterisation. Payment in a tokenised instrument does not alter the recipient's local income tax position or the platform's reporting obligations, and may complicate valuation and withholding. The engagement's characterisation as contracting rather than employment remains a separate and material risk borne by the client business and the platform.

Data protection. The arrangement processes personal data of individuals across many jurisdictions with divergent regimes and cross-border transfer restrictions. On-ledger records are

pseudonymous but linkable, and are not practicably erasable; identifying data must therefore remain off-ledger and, where required, in-jurisdiction.

Realisation by Deel and Tempo

Deel operates a global payroll and contractor management platform serving more than 40,000 businesses and approximately 1.5 million contractors across more than 150 countries. In June 2026 it launched a dollar-denominated wallet for contractors, initially in Argentina, and by August 2026 had extended it to more than 80 countries across Latin America, Africa, the Middle East and the Asia-Pacific. Deel's stated motivation matches the problem above: it reports that 84.6% of its Argentine contractors elected in 2025 to be paid in USD rather than pesos, and that the peso lost more than half its value against the dollar in a single year, while local restrictions capped how much foreign currency workers could acquire.

The instrument is DLUSD, a USD-backed stablecoin issued by Bridge through its Open Issuance platform, with reserves and redemption designed to track USD one-to-one. Wallets are provisioned by Privy and created automatically for any contractor who has completed identity verification on the Deel platform, which is what ties the on-chain eligibility to the off-chain due diligence. Tempo is the exclusive settlement network for the wallet stack, the token, and the earn product above it.

Three network properties are cited as decisive, and each maps to a constraint stated above. Native account abstraction supplies passkey authentication, transaction batching and fee sponsorship at the protocol level, so a contractor authenticates by device biometric and never handles a private key. Fee sponsorship allows Deel to pay transaction costs on the recipient's behalf, so the recipient never needs to hold a separate network asset. Fees are fixed at 0.001 USD, denominated in stablecoin, with no storage deposit charged for keeping an account in state, the combination that makes a very large number of very small balances viable. Deel additionally states an intention to use the network's private zones to keep contractor balances and payout history confidential from public view while preserving auditable access for itself, which is the confidentiality posture described above; this component is prospective rather than deployed.

The yield feature is structured to satisfy L9. Return is generated by Morpho vaults deployed on the network and curated by a third party, not by the issuer or the platform. Opt-in is explicit, there is no lock-up and no minimum holding period, and withdrawal is available at any time. Access to the vault is gated by closed-loop allowlisting enforced at the network layer, so that only verified contractors can route funds into it, an instance of the eligibility enforcement required above being implemented as a token-layer control rather than an application check. Exit paths are redemption to local currency within the platform, transfer on-chain as USDC or USDT, or card spend against the balance.

The result is a single settlement network carrying the whole loop: businesses fund payroll in stablecoins, contractors receive into an embedded wallet, hold as DLUSD, optionally earn, and spend by card, with no bridging and no intermediate conversion. Deel and Tempo report that 74% of payout recipients deposited into the earn product, 60% of eligible users were actively earning, 30-day retention in earn was 85%, and cohort earn balances grew 72% over thirty days, driven principally by net deposits rather than accrued reward.

Two observations bear on the architecture. First, the operator trust model that is unacceptable in the wholesale case (a single party with full visibility into a private partition) is the correct model here, because that party is the platform the recipient already relies on for payroll and which is already the regulated entity. Second, compliance is enforced as an on-chain attribute of eligibility rather than as an exchange of attested outcomes between supervised institutions. Both are configurations of the same partitioning primitive, differing in who operates the partition, who may join it, and what the operator is permitted to see.

Requirements

1. Cross-org & cross-border fast settlement layer
2. Sovereignty, and control (jurisdiction)
3. Privacy (not leaking transactions, but also data protection)
4. Compliance (requirements vary substantially per jurisdiction, e.g., SOX2, gdpr ...)
5. Counterparty risk (sanctions entities, selected validators)
6. No lock-in, open source, common standards
7. Not immutable (freeze, reverts, ...)
8. Deterministic finality
9. No-crypto: fees can be paid in stable coins
10. Fees that are fixed/non-volatile/predictable
11. DeFi, liquidity, composability
12. Interop with rest of the world (web2, other chains, ..)
13. Operational compatibility. Any solution must be as easy as possible integratable with existing systems. This includes changes to how the protocol works and how common APIs are used.
14. Verifiability and recourse. An L2 validator (or user, depending) must be able to prove that a state committed on the L1 was incorrect and that the other validators misbehave

High-Level Design / Design Overview

We focus on how transactions are submitted to an L2 from EOAs, on how interactions (calls) from a smart contract on one chain (L1 or L2) to a smart contract on a different chain (L1 or L2) happen, and on how the fees for this are paid. We do not (yet) describe how an L2 is set up and how its configuration can be changed. Currently, the document does not yet describe how atomic swaps and cross-L2 token transfers are realized (they seem to be a higher-level concern and should be solved at the smart contract level; however, the precompile needs to offer some

support here). We assume that during setup, an account for the L2 is generated on the L1. The authentication mechanism for this account is the one specified for that account, e.g., a threshold of signatures from the L2 validators or a zk proof (this would be supported by the new flexible authentication mechanism that we are specifying separately; in the meantime, one would have to use threshold ECDSA, swallowing the interaction required for it).

The design makes use of precompiles running on all chains that smart contracts can use to make message calls to smart contracts on other chains. Unlike calls in the native EVM, these messages/calls are asynchronous.

Users (EOAs) send transactions (external transactions) destined for an L2 to the L1. These transactions are sequenced by the L1, which creates a virtual block for the L2 using the precompile. An L2 block can only be created if the L2 has executed all of its blocks up to some defined execution gap (this logic is managed inside the precompile). This creates back-pressure on external transactions: if the L2 stalls, L2 transactions stop being included by the L1. When an L1 block is executed, the transactions for the L2 are processed by the precompile. This simply means that the precompile stores the transactions and at the end of the execution of the L1 block, creates all L2 blocks. The precompile can fail a transaction if it has already stored too many messages for an L2 in its queue. In general, the precompile should prioritize internal messages (those generated on the L1 or on L2s) over external messages. When a token transfer transaction fails, the transfer is reverted by the protocol. There can be custom logic allowing the precompile to reject transactions from certain blacklisted addresses (to prevent spam) or only allow those coming from a whitelist (to implement access control). Initial versions of such black- and whitelists are part of an L2's configuration. The precompile can also be configured to delay messages generated by L1 or other L2s if L2 sequencing is to be done off-chain.

As messages to an L2 will typically be encrypted, the validity checks that the L1 does at consensus and in the precompile execution on the L1 can be done are limited to replay prevention and gas and format checks. Checks like “can a certain party cover a value transfer from their L2 account” are forgone because the L1 does not know L2 balances.

L2 validators create *L2 execution blocks* by reading the *L2 consensus block* produced by the precompile on the L1 (posted in the L1 executed block as an event) and considering the L2 state (e.g., scheduled messages on L2). Once an L2 execution block is formed, the transactions in the block are executed and a new L2 state root is computed. The state root is salted to ensure that it does not leak information about the state. The state of the precompile (the one running on the L2) is of course contained therein and includes a Merkle tree of the call messages that the L2 smart contracts want to be sent to smart contracts on other chains (the L1 or other L2s).

The L2 validators then send the state root and each call message as separate transactions to the L1, targeted at the precompile on the L1 (these may also be batch-sent as a future optimization). These transactions are authenticated by the L2 (cf later discussion on how this is done), and their gas is paid from the L2's account (on the L1).

An L2 block is considered *processed* if the L1 precompile has received 1) the state root and 2) all the call messages related to that state root. The nonces generated for these L1 transactions by the L2 ensure that the state root and the messages reach the precompile in this order (cf. how L2 messages are authenticated below). The precompile processes these messages as follows (one by one, as they are all separate transactions):

- If the message targets an L1 smart contract, the precompile simply calls the L1 smart contract with the message (i.e., executes it).
- If the message targets an L2 smart contract, it calls the corresponding precompile (possibly itself, depending on the implementation) to deliver the message. Note that the precompile can fail a message, e.g., because its sender is blacklisted or because there is no longer room for further messages, creating back-pressure.

If a message has failed, the precompile creates a failure reply message to the sending smart contract. Such messages will be part of the next block and should always be included in the next execution block.

We now shed some more light on gas management for transactions involving L2s. As stated, all fees are paid on the L1. When an external transaction is submitted, its gas is paid for by the sender, where the gas includes that cost incurred on the L1 to send the transaction to the L2 and all later costs. These later costs include the processing on the L2 but potentially also subsequent calls to smart contracts on the L1 and L2. The L2 will keep track of which transactions have paid how much and charge them for L2 execution. When the transaction causes a message to be sent to a smart contract on another chain, the L2 attaches the all gas left to that message (the precise details of how gas is attached is somewhat involved and specified below). The base fee of transactions can be different from when Alice had submitted the original transaction. So if Alice has put gas on a transaction that causes lots of cross L2 interactions, the base fee could be much higher and cause Alice's transaction to run out of gas. For MVP, we assume that Alice puts sufficient gas on the transaction to ensure that the processing of the transaction (i.e., all subtransactions) have sufficient gas. Post-MVP, we want to have a gas sponsor design that covers the L1 fees and manage L2 fees separately and a more elaborate fee design overall.

The L2 can regularly reimburse its validators with the gas it has charged for execution (as L1 transactions from the L2's account to the accounts of its individual validators).

Authentication of messages sent from an L2

The transactions that the L2 sends as a result of executing incoming transactions need to be authenticated by the L1 for the L1 to accept them. These transactions either contain a new state root or (a bundle) of call messages from the L2. More precisely, the L1 needs to be convinced that the transaction is a result of the execution of the (latest) L2 block. Depending on how the L2 execution happens, the authentication can for instance be a (zero-knowledge) proof of correctness of the computation, a TEE-attestation of the same, or a multi-signature in case of "correctness by replication."

There are essentially two ways to achieve this and they differ where the authentication is verified:

- 1) Authentication is verified at consensus. Here, the mental model is that the transactions are sent and validated by the L2. To this end, the validators (jointly) compute the authentication artifact (e.g., multisignature or ZK proof) and submit the transaction and the authentication artifact to an L1 proposer. The authentication artifact is then verified w.r.t. the EOA authentication mechanism as specified in the L2 account.
- 2) Authentication is verified at execution of the precompile. Different from case 1), one (or more) validators wrap the L2 transaction and the authentication artefact into a new transaction and submit the new transaction on their own behalf.

We discuss the pros and cons of both mechanisms. Consider the first solution. Depending how the computation of the authentication artifact is done, additional latency can incur (this would be the case if authentication entails a multi signature). Such latency can be removed if L1 proposers take on part of the computation of the authentication artefact, and can be an optimization. In case the authentication artefact is a ZK proof or at TEE attestation, the verification of the artefact requires to have the previous state and the L2 input block available authentically at validation time. This is in general possible because both are the result of a computation of the precompute of the L1 and the L2 cannot execute without that computation having been done (unless a proposer is not a full node). However, one can also just verify the ZK proof or TEE attestation as authentication at consensus and then later reject the transaction in the precompile when the previous state and the L2 block are guaranteed to be available.

The drawback of this is that a fraudulent validator can produce a valid proof for a fake block. If one is concerned with this attack (which can be handled out of band anyway), the authentication for the L2 account can be set such that L1 gas payment is to be by a different account, this concern can be completely mitigated (at the cost that the validators have to have gas accounts and manage them, which they would have to do in the second solution regardless). The advantage of this solution is that the L2 execution can determine the nonces of all transactions and deduplication and replay prevention can happen before consensus.

Consider the second solution. As verification of the L2 authentication happens in the precompile, one needs to send more information through consensus. For instance, if the authentication is by multi-signature, then sufficiently many validators need to send their signature and all transaction artifacts through consensus. Furthermore, the validators need to pay L1 gas for all messages from their accounts.

Comparing the two: The first solution is conceptually cleaner as it treats the L2 as a party that acts on the state and L2 blocks and reports back. It also puts all authentication at the consensus layer, ensuring that no un-authenticate messages use bandwidth. It also means that the L1 precompile can be completely oblivious to how the L2 is implemented (and treating full embedded and private L2s in the same way). Its drawback is that it requires the proposer to have access to the L1 execution state, but as already said, the verification of the correct state being used can be postponed to the precompile. (One might argue that it requires expensive authentication verification at the proposal and block validator level, but that argument breaks down because 1) the work needs to be done regardless and 2) invalid authentication artifacts can be somewhat more costly to reject, sending invalid messages is ultimately a problem that

needs to be dealt with at the level of spam prevention.)

The second solution will generally incur more consensus bandwidth, requires the L2 validator to manage their own gas accounts, and requires additional authentication logic in the precompile that does not belong there conceptually.

Being conceptually simpler and more elegant, we choose the first mechanism.

Design decisions for MVP

- We do not time out messages for now, as this can cause all kinds of race conditions. All we do is that we limit the number of messages that we buffer in the precompiles. If the buffer is full, the precompile will fail messages. This means that funds can be stuck forever in the input buffer of an L2 if the L2 no longer processes blocks. However, the damage is bounded, as after the buffer is full, the precompile will stop accepting messages.

Access control to L2 state / Authenticating eth_calls

As the L2 state is no longer public, access to it needs to be controlled. Thus, state can only be accessed via authenticated eth_calls. Unauthenticated calls are blocked and other queries are blocked. That is, messages for eth_calls need to be handled as if they were replicated transactions but ignoring the gas_fee parts. This is similar to what Tempo zones RPCs do (cf [tempo zone rpc design](#)).

This means that:

- Smart contracts (need to) implement access control, e.g., an account can only read its own balance but no other balances.
- Error messages in face of failed access must not provide failure detail to address side-channel attack.

Set up (draft-tbd)

Setting up an L2 will (somehow) create the following,

- A list of validators that operate the L2 (identities and signing public keys).
- An L2 account with authentication set such that messages sent by the L2 validators can be authenticated as coming from the L2. This includes a threshold signature scheme and in the future can include TEE authentication and ZK proofs.
- An encryption public key of the L2 such that messages to the L2 can be encrypted. In the MVP, all L2 validators have a copy of the secret key.
- Initialize the precompile on the L1
- Creation of the initial state of the L2, initialize the precompile on the L2, creation of ERC-20 L2 smart contracts for the wrapped tokens

Verifiability and recourse (TBD)

- Describe the mechanisms that allow 1) zkVM and TEE to prove they did things correctly and 2) in case of committee L2 validators to prove that a new state root was not computed correctly.
- What does need to be stored on-chain and what in DA for this to work?

Natively wrapped tokens (draft outline)

- L1 tokens on L2 are wrapped natively for the tokens specified during setup
- User balances on the L2 and on the L1 are different. This is necessary for security, otherwise the L2 would have control over user funds on the L2. When a user transfers tokens on L1 to L2, the tokens are transferred to the L2 account with the memo being the user's account. This will create a minting event on the L2 contract and the minted L2 tokens get sent to the user's account on the L2. When a user sends tokens from an L2 to another L2, the sending L2 burns the L2 tokens, transfers the native tokens to the account to the receiving L2, which will then mint tokens, and credit the correct receiving account. If the receiving L2 has the token not natively embedded, the precompile on the L1 will fail the transaction, causing the funds to be returned.

Data availability (tbd)

Cross L2 payment (need to ensure that tokens cannot get stuck, i.e., if receiving L2 rejects the message or the message fails, then the token can be claimed by the sending L2 and sent back to the sending user).

- Cross L2 swap

Specification

Setting up an L2. The L1 maintains evidence of the different L2s and the identity of the respective validator sets and RPC nodes for the L2s but never directly reaches out to the L2 nodes. Registering an L2 is done by specifying the identities of the respective nodes that will run it. Ensuring those nodes are actually set up and agreed to run the L2 has to be done in advance and enforced through off-chain contracts (e.g., contracts between institutions and node operators). We provide playbooks making setting this up very easy once the respective hardware is available. These playbooks will:

- Install the modified client implementation needed to run the L2s, together with all required privacy-related extensions;
- Set up communication between the L2 nodes and the L1 nodes.
- Set up communication between the L2 nodes and each-other. This is needed in sidecar protocols allowing the L2 nodes, e.g., to perform state synchronization for the L2. Additional sidecar protocols can exist to optimize communication between the L2 nodes.
- For RPCs: install the required serving code.
- Last but not least: validate that the whole setup has been done correctly and registration is ready to proceed, informing of any faults identified in the process.

Communication model. Communication is essentially one directional: $L2 \rightarrow L1$, never backwards. In particular:

- $L2 \rightarrow L1$ is push-based (the L2 validators submit to the L1 validators);
- $L1 \rightarrow L2$ is pull-based (the L1 advertises information as part of L1 blocks, which the L2 validators listen for).

Star topology. Communication between the different namespaces (i.e., between the L1 and the different L2s) follows a star topology with the L1 at the center: all cross-namespace messaging is routed through the L1: no two L2s can communicate directly without passing through the L1. This is the main idea required to realize the promised robustness of messaging.

Security & Liveness model. The security model is that each L2 trusts itself and the L1, but the L1 only trusts itself. As a result, only the L1 is assumed to be live, the L2s can stall arbitrarily if the respective nodes running them go down.

Asynchronous calling model:

- **Messages.** The EVM works at the level of *calls*, also known as *messages*. The two names are faces of the same coin: the former models the interaction from a

remote-procedure-call point of view (smart contracts can call each-other), while the latter models it from a message passing view. A call may internally trigger downstream calls, e.g., a smart contract calls another, which then calls another, and so on. Equivalently, in the message passing view, the first contract sends a message to the second, which processes the message and sends another message to the third, etc.

- **Transactions.** *Transactions*, on the other hand, wrap a top-level call/message and all its downstream execution. The name transaction suggests atomicity: the whole call tree executes as one atomic unit. With the present feature, transactions remain atomic and execute in a single namespace, but the system gains asynchronous calling capabilities. In particular, within the context of an executing transaction, an asynchronous call to another namespace can be fired. The corresponding call executes in a different transaction in the receiving namespace. Unlike synchronous calls, which return a value to the caller upon finishing execution, asynchronous calls are “fire-and-forget” from the perspective of the caller:
 - The call receiver may fire back a message to the original caller, allowing it to act on the outcome. De facto, this permits implementing async-await/request-response style semantics at the application layer.
 - The system *may* ensure such messages are fired back automatically. This allows, e.g., automatic bouncing back of rejected token transfers. Not needed for the most minimal design, but required for actual MVP promises.

Precompiles: The feature is primarily driven through precompiles. Each namespace **N** has a precompile **N.P** exposing different callable APIs:

- **User facing:**
 - On all namespaces, the precompile exposes an API for firing asynchronous calls to other namespaces: **N.P.remote_call(target_chain_id, <msg>)**.
 - Only on the L1 namespace, the precompile exposes a second API for asking an L2 transaction to be added to the L2 sequencing pool:
L1.P.send_L2_tx(target_chain_id, <tx>).
 - Both APIs may reject, e.g., for rate limiting reasons or if the respective message or transaction is ill-formed.
 - Note that the two functionalities serve different purposes, despite being similarly flavoured.
- **System facing:**
 - Only on the L1 namespace, the precompile exposes additional API methods to be used in L2 block building, L2 state roots settlement, and cross-namespace message processing. These are only expected to be called by the L2 operators or the system.

L1 precompile: The **L1.P** precompile manages block building and settlement, as well as asynchronous message processing for all the L2s through an internal state machine.

Concretely, for each L2 namespace `chain_id`, it maintains the following data structures that are interacted with through corresponding APIs:

- A list `pending` of pending call-messages and transactions addressed to `chain_id` that have not yet been included in an L2 block:
 - Pending asynchronous calls get appended to the list via the first user-facing API mentioned above `L1.P.remote_call(chain_id, <msg>)`.
 - Pending L2 transactions get appended to the list via the second user-facing API `L1.P.send_L2_tx(chain_id, <tx>)`.
- A list of the produced `chain_id` L2 blocks, in order.
 - Every time the precompile produces a new L2 block for `chain_id`, it appends it to the list and emits a `NewL2Block(chain_id, <payload>)` event, visible in the executed L1 block.
 - L2 validators listen for such events on the L1 chain and pull the respective blocks to start executing them.
 - Triggering the production of new L2 blocks is done through a special system API `L1.P.ProduceNewL2Blocks()`. The details of how exactly this works are quite flexible:
 - The system may append a transaction calling `L1.P.ProduceNewL2Blocks()` at the end of each L1 block. This may take round-robin-style approaches to ensure no L2 is starved.
 - Once it has been decided which L2s should produce blocks in this L1 block, the actual block-building strategy will iterate over `pending` and decide how to order before committing to an ordered block. It can also decide to postpone some of `pending` for a later block or to discard some of it (e.g., if there is a nonce gap). The exact logic can be specified by the L2 operator within some fairness bounds, details **tbd**.
 - Alternatively, we can let an L2 centralized sequencer call `L1.P.ProduceNewL2Blocks()` directly, specifying the order they want.
- For each produced L2 block `B`, the precompile `L1.P` maintains two-initially empty fields, `B.state-root` and `B.msgs-pending-count`.
 - The initial (`empty, empty`) state corresponds to `B`'s execution not yet having been settled on the L1.
 - The L2 validators, after executing an L2 block, call an API `L1.P.deliver_proof(chain_id, B.block-id, state-root, msgs-pending-count, <proof>)`. If the proof verifies, the state of the precompile updates `B.state-root` and `B.msgs-pending-count`, setting them to the given values.
 - At this point, `B` is considered successfully *executed* but not yet fully *processed*: there are `B.msgs-pending-count` many outbound asynchronous calls from `chain_id` that executing the block has produced. These have to be delivered to the L1 and processed before the block can be considered processed.

- One by one, the L2 validators submit the missing transactions by calling yet another API: `L1.P.process_inbound_msg(chain_id, B.block-id, <msg>, <proof>)`. The proof here is a Merkle proof proving that the given message was actually fired as a result of executing block `B`, i.e., a proof against `B.state-root`. If the proof verifies, `B.msgs-pending-count` is decremented, and the message is *processed*, which depends on the target chain id of the message:
 - If the message's destination is the L1, then it's immediately executed as a normal EVM call/message.
 - If the message's destination is another L2, then the `L1.P` self-calls into itself as `L1.P.remote_call(msg.destination_chain_id, <msg>)`. We call this process **rerouting** the message.
 - Future versions may provide an interface to batch submit multiple pending messages at once.
 - We can enforce in-order delivery of messages easily. Left out of the specification for now for clarity of exposition.
- The previous method `L1.P.process_inbound_msg()` can (and probably should) enforce *in-order delivery*. In other words, the asynchronous calls fired by each L2 are processed in order on the L1. This is achieved by failing a message delivery request against a settled state-root in the `L1.P` if the previous message in the sequence has not yet been delivered: this requires messages sent by each L2 to have a sequence number and a small amount of additional bookkeeping logic in `L1.P`.
- When `B.msgs-pending-count` hits 0, `L1.P`'s state machine considers the L2 block `B` to be *processed*. Note how this is strictly stronger than *executed*, which merely means that the state root has settled on the L1.

Backpressure in the L1 precompile: The role of the produced → executed → processed state transitions for each L2 block is to allow implementing a backpressure mechanism: we want to stop L2 block production if the L2's executing validators don't deliver or experience delays. This is achieved by two simple rules:

- The user-facing APIs: `L1.P.remote_call(chain_id, <msg>)` and `L1.P.send_L2_tx(chain_id, <tx>)` on the L1 reject the call if the difference between the index of the last produced L2 block for the given chain id and the last *processed* block for that chain id grows beneath a certain threshold, like 5 or 10.
 - These are synchronous rejects: you get to see immediately whether the call succeeded or failed.
 - Note that even without stalling, these methods may also reject in high-demand situations, i.e., when the number (or total gas) of the messages queued for sequencing in `L1.P.chain_id.pending` has exceeded a certain amount.

- There should likely be some fair preferential treatment for cross-namespace messages over normal L2 transactions: if those fail due to rate-limiting of the destination namespace, the impact is higher.
- Additionally, the block-production logic in `L1.P.ProduceNewL2Blocks()` will not allow producing any new L2 blocks for the given stalled L2 until the stall is recovered from.
- This protects the users by rejecting calls and transactions targeting stalled L2s and also protects the system from having to store an unbounded number of “opened-but-not-closed” L2 blocks.

The L2 precompiles: For each L2 namespace `N`, its precompile `N.P` also implements a state machine, albeit considerably simpler:

- It only exposes the single API introduced above `N.P.remote_call(target_chain_id, <msg>)`.
- Internally, it stores a list `N.msgs_sent` of pending calls fired to other namespaces.
- Naturally, the method can synchronously reject new calls if too many such calls (or with a too-high total-gas limit) are made in the same L2 block’s execution. Whenever a new call is accepted, the precompile emits an event `MsgSent(N, <payload>)`.
- At the end of each L2 block execution, the L2 validators compute the new L2 state root and a proof of correct execution, which can be multisig, ZK, etc. They then send an L1 transaction targeting `L1.P.deliver_proof(N, B.block-id, state-root, msgs-pending-count, <proof>)`.
- Afterwards, they proceed sending the messages sent in this block (inspecting the emitted events or otherwise), one by one (or in a future version, batched) as individual L1 transactions targeting `L1.P.process_inbound_msg(N, B.block-id, <msg>, <proof>)`. The proofs are the Merkle proofs of inclusion of the respective messages in the just computed state root.

Transactions: in the flows above there are multiple details to settle in terms of:

- How do messages materialize into transactions for the purposes of being included in L2 blocks?
- How is gas managed across inter-namespace calls?
- When is gas paid for and by whom?

Most importantly, gas payment is now decoupled from gas accounting. The flow of gas itself and payment for gas are now highly decoupled. All of these details interplay.

Gas accounting and payments for work: All gas payments are done on the L1.

TODO:

- Gas accounting (for now single gas account to send from by the L2; all fees paid on the L1, out of band payment to L2 validators, authorization questions before transactions hit the L2 following Mike's comment).
- Calls model, callbacks.

Followup notes on “Modes the L2 validators submit transactions back to the L1” discussion

Andrei Constantinescu Kushal Babel Jan Camenisch :

- For multisig authentication of execution state roots, we get the ideal scenario: there is a single gas account of the L2 on the L1. The L2 validators compute a multisig and then submit it to the L1. Consensus can check the validity of this proof (as part of account authentication) before putting it into the consensus block: spending funds from this gas account is guarded by a multisig. Moreover, if multiple L2 validators submit the transaction (which is required for fault-tolerance), it will only be sequenced once, as they will all have the same nonce.
 - Very fine print after the call (very inconsequential): note that the fact authentication passed does not mean the state root will be accepted: it most likely will, but the precompile will still reject a state root for an inexistent block (a basic sanity check that can only be done in execution). This can happen only in majority corrupted cases, and there is nothing to do against that, but it prevents the precompile from getting into an illegal internal state.
- For ZK, we had two competing solutions, and a third that solves their shortcomings:
 - We want all L2 validators to use the same account (all of them hold the key for it) so that if multiple of them submit a ZK-bearing transaction, the nonce prevents it from being sequenced twice.
 - We would also want the L2 validators to submit from their own gas accounts so that they cannot drain the L2 gas account with correct but for-the-wrong block transactions that ultimately fail the “does this block exist” check in the precompile (we can validate the proofs in consensus, but they may be for the wrong state, and with asynchronous execution consensus cannot know). There could be a flow where L2 validators get reimbursed if the precompile check passes. However, this brings back the nonce problem: consensus may include multiple such transactions, as they come from different accounts.
 - The proposed solution from the call: let the transactions be submitted from the L2’s gas account (so single nonce sequence) but as a new kind of authentication transaction requiring the sender (L2 validator) to sponsor the gas. This way: it’s the L2 validator’s gas money, but the transaction still comes from a single nonce sequence so we won’t include more than one good transaction per block.
 - Now the sad part: you can’t actually do deduplication this way..., consensus will only check the ZK proof, but not the “does the state exist” part. If two L2 validators submit different ZK proofs, consensus has to choose one, and it might be one is good and one is bad, and only execution will tell... With bad luck a bad L2 validator can prevent a good one from being included based on the nonce (as consensus will only keep one).
- **Conclusion so far: problem is real.**
- New requirements:
 - If people want to send the same proof 5 times, it gets deduplicated.

- If they want to send different proofs, it does not get deduplicated, and execution does its job in picking the "correct one".
- Nonces are not the solution for this, but there can be hash-based solutions for this.

DRAFTS, DON'T READ

[To make the presentation easier to follow, let us assume there are three namespaces, termed the L1, the L2.1 and the L2.2 (identified by different chain_ids).]

TODO for tomorrow:

- All L1 traffic through precompiles.
- Precompile produces blocks posted as events.
- L2 pulls those and executes.
- Posts back state roots.
- Precompile decides if a new block may even be produced.
- Asynchronous difference: proving L2 blocks vs providing their messages out of band.
- Async calls are “detach thread()” style.
- Semantics of calls, gas forwarding.
- Types of transactions considered, some don't pay gas, but gas is always forwarded.
- Precompile L1 methods: `sendMsg()` vs `sendTx()` vs `provideProof()` vs `runMessage` against `root()`.
 - Data structures here.
 - Off-chain sequencing support ergonomics vs `ProduceBlock()`;
 - Consensus checks.
- Precompile L2 methods: `sendMsg()` with throttling, that's it.
 - Validators compute the final state root, with maybe a fast-path to the messages just sent; then L1 gets this and we can begin proving against it.
- Privacy/encryption requirements and what parts are encrypted:
 - Recipient always encrypted against the public key of the destination chain-id (identity encryption function for source).
 - Nondeterministic, formally:
 - semantic security, and equivalently IND-CPA (indistinguishability under chosen-plaintext attack)
- Consensus backpressure.

Their nonces are generated by the L2 (general discussion on replay protection).

Jan sends an L1 transaction with `gas_limit` = how much work we do on the L1 and an envelope that say:

“Send this money from Jan to Andrei, this downstream message needs `downstream_gas_limit` = ...”

Precompiles, :

- **Public:** all namespace precompiles provide an API for firing asynchronous calls `N.P.remote_call(target_chain_id, <msg>)` that can be called by executing transactions to make an asynchronous call `<msg>` to another namespace
`target_chain_id`:
 - The method either succeeds or fails, informing the caller synchronously. Failing will mainly happen for rate-limiting reasons.
 - If the method succeeds, the remote call is considered *enqueued*. It is then the responsibility of a different mechanism to ensure the call is eventually delivered to the target namespace and executed in a transaction there. We specify the exact flows later, as they differ across: (1) L1->L2 calls, (2) L2 -> L1 calls, and (3) L2 -> another L2 calls.
- **L1-only functionality:** the L1 precompile `L1.P` offers an additional API:
 - `L1.P.send_L2_tx(target_chain_id, <tx>)` → unlike `remote_call()`, which was a request to send an L1 → `target_chain_id` message, this is a request to run an internal L2 message. Depending on security tradeoffs we want to make, `<tx>` can be completely encrypted, or only partly encrypted.
 - As before, the call may fail, mostly for rate-limiting reasons, e.g., if the `target_chain_id` has run behind in processing execution blocks.
- (2) It contains special “here is an execution proof for a given L2 block” txs.
- (3) It contains special “Deliver a message for this L2 executed block” txs.

In particular, each transaction wraps a top-level call to be executed. This call may internally

Transactions vs. messages. We distinguish two related but very different concepts:

1. **Transactions** `tx` in the classical sense. These are keyed by a `chain_id` targeting the namespace they are aimed at, i.e., `L1`, `L2.1` or `L2.2`. Transactions always do work in a single namespace, never cross-namespace, but can trigger work to be done asynchronously in other namespaces, see below.
2. **Messages** `msg` on the other hand, cannot be input directly, and are instead created during transaction execution. They have an `origin` namespace (the one whose execution context produced them) and a `destination` namespace. They also have a `payload`, which when triggered performs work on the destination namespace. The payload can only be triggered within a transaction in the execution context of the destination namespace.
 - a. E.g., an `L1` transaction can say “first, trade on this DEX, then bridge 10 MON into `L2.1`”. This transaction will perform the DEX trade leg and then enqueue an outbound `L1` → `L2.1` message saying “Hey, please update this User’s balance”, which is work to be performed on `L2.1`.

Note: if it helps, we might want to separate “origin” (who created the message) from “sender”: who is the latest relayer.

Sequencing. All transactions are sequenced by the `L1`, i.e., in `L1` consensus blocks. In particular, `L1` consensus blocks will contain three separated sequences: one for `L1` transactions, one for `L2.1` transactions and one for `L2.2` transactions.

Routing and Execution Flows. Basic messages, i.e., those where either the origin or destination namespace is the `L1` go through three conceptual stages:

- **Sent:** committed to the outbox of the sending namespace. At this point, the target namespace is unaware of the sent message;
- **Received:** the target namespace has received and committed the said message. So far, its payload is completely ignored, just stored on the target namespace;
- **Executed:** a transaction on the receiving namespace has activated the payload of the message.

For cross-L2 messages, where the origin and destination namespaces are both L2s, messages go through five stages:

- **Sent** by the origin L2;
- **Received by the L1;**
- **Resent by the L1** to the destination L2;
- **Received** by the destination L2;
- **Executed** by the destination L2.

In other words, delivering messages to their target namespaces and triggering their payloads follows prescribed paths along the star topology:

- **L2 → L1 messages.** E.g., **L2.1 → L1**:
 - They first land in **L2.1**'s outbox (in **tx1**), then in **L1**'s inbox (in **tx2**), then are executed in the **L1** namespace (in **tx2** or a separate transaction **tx3**).
- **L1 → L2 messages.** E.g., **L1 → L2.1**:
 - They first land in **L1**'s outbox (in **tx1**), then in **L2.1**'s inbox (in **tx2**), then are executed in the **L2.1** namespace (in **tx2** or a separate transaction **tx3**).
- **L2 → L2 messages**, say **L2.1 → L2.2**:
 - They first land in **L2.1**'s outbox (in **tx1**), then in **L1**'s inbox (in **tx2**), then in **L1**'s outbox (also in **tx2**), then in **L2.2**'s inbox (in **tx3**), then finally they execute in the **L2.2** (in **tx3** or a separate transaction **tx4**).

Note how executing the payload of a message can but need not happen in the same transaction delivering it to the destination namespace.

Precompiles. Within each namespace **A** (the **L1** namespace or one of the **L2** namespaces) lives a single stateful *inter-namespace communication* precompile **A.Comm**. This is primarily responsible for:

- Maintaining **A**'s inbox (messages originating in other namespaces that **A** must process) and outbox (messages that **A** addressed to other namespaces to handle).
- Providing an API **Comm.send(message)**. In particular, a smart contract can call this to add a message to the current namespace's outbox. The **send()** method can fail under certain circumstances, informing the caller.
- Providing an API **Comm.receive(messages_list)**. The **receive()** method should only succeed if the given list of messages were actually meant to be received by this namespace. In particular, an **L2** directly sending messages to another **L2**...

The precompile is the view of the namespace on other namespaces: **L1** it sees messages from/to **L2**'s and for **L2**s it sees messages from/to the **L1** (and transitively from other **L2**s).

This should go...

Lifetime of a cross-namespace message in the happy case:

Say namespace **L2.1** wants to send a message **msg** to namespace **L2.2**. In particular:

```
msg = { .origin = L2.1,  
        .destination = L2.2,  
        .payload = ... }
```

Then, the happy case flow is as follows:

- In namespace `L2.1`, some transaction calls `L2.1.Comm.send(msg)` and the message is accepted for sending. This message appears in the outbox of namespace `L2.1`. For now, only `L2.1` is aware of the existence of the message.
- `msg` is included in an L1 transaction targeting the `L1.Comm`.

Precompiles. The view each namespace holds for other namespaces is governed by a precompile living in the respective namespace. In particular, there is a precompile in the L1, and one on each L2 namespace. Given the star topology and asymmetric communication model, the L1 precompile holds different information than the L2 precompiles.

Queues. The L1 maintains a sequence of messages sent to each L2. Similarly, it maintains a sequence of received messages from each L2. And also vice-versa: each L2 maintains a sequence of messages sent to the L1 and one received from the L1. i.e., if there are n L2s then there will be a total of $2n$ sequences. For now, these are sequences (there is no garbage collection of old messages), in a later version they can become queues. At the implementation level, these sequences are stored in stateful precompiles.

Operation of the L2:

L2 block creation: the validators of the L2 listen for the L1 chain. With each L1 block `Bn`, they pull in the following information: the transactions targeting the given L2 present in `B`, call them `X` and the messages sent or routed in the L1->L2 queue during `Bn`'s execution, call them `Y`. Together, `X + Y` constitute the L2 block `B'n`.

L2 block execution: when the L2 executes the block, it executes the normal transactions and appends the respective messages to the L1->L2 queue of received messages. It may additionally append messages to be sent back to the L1.

Sending back to the L1: after executing the L2 block, each of the L2 validators computes the new state root of the L2 and an ordered sequence of newly sent messages to be routed back to the L1 and sends these as part of L1 transactions addressed to the L1. In particular, these transactions have the shape "I am an L2 validator, I finished executing `B'n`, here is the new state root (perhaps together with a proof of this fact in the ZK-setting), and there are the new messages I have ready for you, L1". Each L2 validator that successfully executed `B'n` sends such a transaction to the L1.

Optimization of the previous: In a later version, the L2 validators can run an all-to-all messaging sidecar protocol to produce a single transaction to be sent to the L1 (this protocol can be shipped together with the modified client to be deployed in the playbook). Sending it only once requires a leader-election-style primitive or a rotating schedule of senders (including what to do when the leader does not send), while

agreeing on a single certificate of a correct transition is a monotonic process not requiring such complications. We can consider implementing such primitives.

Operation of the L1:

Executing the special transactions sent by L2s: these transactions target the precompile on the L1 and indicate that the respective L2 validator has executed the respective L2 block and produced a new state root + messages to be delivered to the L1. The precompile ingests these proofs and only modifies the state of the respective inbound queues + advances the state of the respective L2 once the proof of advancement has been accepted (for ZK, a single proof suffices, for multisig multiple of these transactions have to go through before the L2 state transition can be considered valid).

Routing on the L1: in executing every block, the L1 will deterministically take some messages addressed to it and “route” them to the right place. In particular, if those messages target a different L2 as the destination, it will place them in the “to be sent” queue for that L2. If the message is addressed to the L1, the L1 will not route it further.

Triggering Messages:

One missing piece above is the way a message goes from “received” into “triggered”. Without additional infrastructure, once a message is received, it needs to wait for yet another user-inputted (or operator-inputted) transaction to go through the L1 and ultimately settle in the right namespace, triggering the message payload. This is inefficient and can be censored. Here the “scheduled transactions” feature comes to the rescue: once one of the state machines of any of the namespaces decides a given message is “ready to fire in a transaction”, it schedules this transaction automatically and deterministically. Note that we do not want to perform work outside of transactions (there was friction in the past).

Contrast with namespaces MIP design doc:

Kai’s doc decouples message delivery even further: messages from one namespace to another first only settle on the destination namespace as a hash/merkle root. It is then the responsibility of future transactions to send the actual messages in transactions (which are validated against the merkle root that is posted together with the state transition proof). Most generally, this basically splits the status of a message into 4: “sent”, “hash posted”, “message delivered”, “message executed”. However, in that design, “message delivered” = “message executed” -> when the actual message is supplied to the executing namespace, it is also executed as part of the same transaction, while the “scheduled transactions” feature allows us to defer its execution.

Old stuff, some of it still valid to be harvested and deleted later - to be ignored

Questions to discuss with Jan:

1. All ingress tx go through the precompile or not?
 - a. Precompile is easier: all done inside, all state of deferred roots stored inside, all L2 block producing logic held inside.
 - i. Can have a method ProduceL2Blocks() that we call at the end of every L1 block's execution. This also implements any permissioning, allowlists etc, and does the ordering, with perhaps custom logic for each L2.
 - ii. L2 blocks can be posted as L2BlockPosted events (appear in the L1 executed blocks). Alternatively, every message to be sent can emit some MsgSent event and L2 validators read this and maybe the precompile state to figure out when a block has been produced (messier, but the idea is still abstracted enough away). **Overarching mental model to be had: L2 validators only read L1 executed blocks** (compatible with your writeup).
 - iii. Problem with this approach: consensus won't be ordering or checking validity of L2 transactions in any thorough way. We can make ProduceL2Blocks() apply ordering logic, but consensus will still feed it with some incorrect transactions (e.g., skipped nonces) if consensus is fully opaque to what the precompile does inside.
 - b. Better but harder alternative: L1 consensus orders L2 ingress before the precompile is hit (so transactions that abort are rarer). The L2 blocks would then be assembled by a combination of reading L1 executed blocks in two ways: one reads L2 ingress, another reads events emitted during the block's execution (for messages).
 - i. Notably, the consensus L2 ingress inclusion checks will have to be quite complicated: in MCP consensus you don't have access to the parent block: there is an execution delay and a new **observation delay**. So, you

can't just say something like: "consensus reads the precompile state" -> it can only read its state from a few blocks back, and doesn't even know what got included in the meanwhile. MCP spec deals with all that for the L1.

- ii. Similar more down-to-earth problem: at the time you build the consensus block, say in round n , you don't even know if in round $n-1$ the L2 operator supplied a valid execution proof for preceding L2 blocks, and you don't even know how many preceding L2 blocks have been produced in the blind window: so you may get a proof of execution of an L2 block you're not aware exists yet (you might de facto be aware, but consensus validity should not depend on the blind window)... that's how annoying it seems to get.
 - iii. This is just to list some of the complexities of deciding at inclusion time what makes sense to include.
 - 1. Probably all of this would be circumvented if we only produced L2 blocks every 10 L1 blocks or so (i.e., enforce that you have visibility for the L1 block that could have produced an L2 block). I think I would be down for some sort of solution like this: it gives up latency but it seems like it can be a mess anyway to get low-latency here with MCP (at least from a preliminary point of view...)
2. Gas payment: so I understand all gas is paid on the L1; part of it is forwarded to the L2's gas account on the L1 that can pay the L2 validators out of band.
- a. **UX: let's discuss how gas_limit is determined on messages.**
 - b. Accounting across successive messaging legs is easy: just subtract with each moving leg.
 - c. I guess when control goes back to the L1, the L1 takes money from the L2 gas account on the L1?
 - d. **Problem: base fee has moved, so what were enough funds to cover in the original block the "atomic tx" started may no longer be enough funds to cover the rest. I think we'll need a different accounting model because the price of gas is different in each block.** Scheduled transactions already get their own accounting with predebiting so we should consider doing something different from standard EVM here as well, but it's not as straightforward as it is there.
3. Two modifications you made, but they are fine:
- a. No explicit msg queues L1->L2: all new pending messages become part of the new L2 block, there is no partial consumption of a queue as we had initially.
 - b. Less delivery vs execution separation in the L2->L1 flow: one idea was that you deliver messages from the L2 to the L1 and then defer their execution alla scheduled transactions: once they are on the L1, we can let them execute over time. You changed this with "messages are committed one by one and executed when received", don't see a real problem with that, other than we may want to add batching to allow L2 operators to send more than one at once (to offset the

inclusion costs of the batches and get better CR-resistance -> every new tx is a new opportunity to be censored).

4. Could be very productive talking about nonces and in general what fields are encrypted in each flow.

Tempo does cross-L2 token transfers -> they get priority (some bandwidth reservation or something).

Block n on the L1:

- It contains normal L1 ingress (ignore)
- (1) It contains special "please sequence this L2 transaction <tx> (blackbox envelope)"
- (2) It contains special "here is an execution proof for a given L2 block" txs.
- (3) It contains special "Deliver a message for this L2 executed block" txs.

When block n **executes** on the L1 the following happens:

- Each type-(1) tx calls into the precompile (we are in precompile execution).
 - Internally the precompile has a lot of things:
 - The ids of previously produced L2 blocks
 - The "were they executed" on those L2 blocks.
 - Waiting space of not yet sequenced messages. (*)
- Each type-(2) tx again calls into the precompile, the precompile updates the "what got executed state".
- Each type-(3) tx again calls into the precompile.

At the end of each L1 block we **inject** a very special transaction that calls Precompile.TryProduceL2Blocks():

- Then this again executes precompile code, looks how far behind we are backlogged.
- At the end it either produces a block and emits an NewL2BlockCreated(<txs list>) event, but also! Modifies the state of the precompile to set "pending msgs = \varnothing" and record the newly created block.

L2 validators watch the L1 chain for NewL2BlockCreated events, and pull L2 blocks from them, and starts executing them.

Certified vs non-certified RPC calls into the L1:

- If you want lower latency you also become an L1 full node.

RPCs also have the full state and they execute the L1.

Access Control

The state of L2 is private by default. This is very different from a normal EVM state which is fully public. Normally, state can be accessed in many ways: downloaded, queried via `eth_calls`, and by smart contracts calling each other. If state is to be private, one needs to add a layer of access control.

Wallet integration

Interaction with tokens on L2 is different than with tokens on the L1:

- Balances cannot be read via normal `eth_call`, but require authenticated calls
- Transactions to an L2 need to be encrypted before signed and submitted
- L1 on tokens are bridged to L2s, the wallet must support
 - Sending an L1 token to an L2 account bridging it
 - Sending the token from an L2 to an L1 account
 - Sending the token from one account on one L2 to an account on another L2.
 - Accounts on L2 might be gated by KYC, so the wallet should first check whether the receiving account is cleared to receive tokens

RPC Integration of L2s

Things to cover (from the whiteboard):

- Backpressure for L2 ingress (to protect L2 and other L2s who might be waiting for it);
 - Queues are bounded in length.
 - Trying to enqueue when a queue is full fails the call.
- Related: callback mechanics without native request-response pairs.
 - Maybe we provide a library.
- Timeouts on the L1.
- Signal for the L2 to know that the L1 has consumed a message;
- Logic of returning funds after a message has timed out;
- Consensus mechanism for sequencing on the L1.
 - This has two interleaved components: ingress goes directly into L1 blocks.
 - During consensus validation we check the L1-precompile state.

- The precompile records the last time a state root was supplied (i.e., block executed) plus the sent vs received state of each inbound queue (to stop producing L2 blocks if a given L2 is not delivering the required messages it has committed to through hashes).
 - **Open:** inclusion rules wrt nonces and confidential state and balances, seems like with encrypted transactions it already becomes tricky.
- Sending of batch proofs: the L2 validators send to the L1 proposer.
 - Ideally, the L1 validator should not be aggregating proofs.
 - Aggregated proofs implicitly multisig on spending the execution gas on the L2's account in the L1.
- Why not “sharding”: “we want to rely on the L1 as much as possible”. Ideally, with TEEs or zkEVM you'd only be running 1-2 executing validators per L2, and this is not enough to fully sequence anything.
- Gas sponsorship can happen by the Bank wrapping user transactions into envelopes (signing them for gas), sort of a la <https://eips.ethereum.org/EIPS/eip-4337> gas sponsorship with user ops.

[DRAFTS, don't read]

L1 block execution:

Since communication L1<>L2 is asymmetric, we describe the two flows separately:

1. Each L1 block contains L1 transactions, as usual, as well as L2 transactions, keyed by the different L2s.
2. The L1 has a precompile maintaining outbound message queues targeting each of the L2s.
3. L2 block construction proceeds as follows: the L2 validators constantly watch the L1 chain and have a predetermined cadence, say $C = 3$ blocks. Every 3 blocks, they observe the L1-sequenced blocks and take the latest transactions addressed at the namespace
4. + observe the L1->L2 queue in the L1 for new inbound messages.

Concretely, say the L1 produced the following blocks, where all txs are to be understood as addressed to this L2:

Block 1:

- message 1
- transaction 1
- message 2

Block 2:

- message 3
- transaction 2

Block 3:

- transaction 3
- transaction 4

Block 4:

- transaction 5
- message4

Then the first L2 block will contain txs 1-4 and messages 1-3.

Embedded and partially embedded L2 (namespace)

“Notes” and followup from Meeting July 16, Andrei, Kai, Mike, Jan

Design goal:

- Setting up an L2 is easy: is done by submitting a creation transaction that specifies the properties of the L2 including the following (submission can be by an L1 smart contract or an EOA):
 - Set of validators that execute the L2 transactions (a subset or all L1 validators)
 - The validation criteria (impacting security and liveness of the L2)
 - k out of n (i.e. if k validators have produced the same state and tx)
 - signature, ZK proofs, etc
 - there might be some bound here that have to be satisfied (don't do n out of n), the main chain should not stall on waiting for L2 input
 - Whether or not the state is private
 - Available to the validators
 - Running inside TEE;
 - MPC, FHE, etc
 - If private: ERC-RPC (in particular eth_call) has access control similarly to tx verification

- List of tokens are available in the L2 (native + ERC20)
 - And policies for them (cf Tempo Zone)
- Access control: list of addresses that are allowed to send transactions to an L2, can be all
 - This allows for sequencing by a dedicated party
- The setup can be changed (by the same party, by a smart contract on the L2, ...)
 - Validators can be removed or added
 - Tokens can be removed or added
 - Modify access control list
- Tokens are a first class citizen and cross L2 token transfers are atomic (but not necessarily synchronous)
- We do not want to rely on relayers or any other external parties for any of this to work. In the rare case of failures, however, intervention by external parties can be necessary
- One might need to have some bandwidth guarantees & limits
 - Token transfers
 - Amount of L2 transactions (there can be a limit here)
 - Could also be bandwidth reservations that L2 pays for

Nice to have

- Gas can be paid in stables
- Gas can be paid by L2's account on L1 (more generic notion of sponsoring gas..)

High-level design for messaging (state commit and tx scheduling)

The L1 execution block contains the transactions that the L2 will execute.

After processing the L2 part of the L1 execution block, the L2 validators have each produced

1. A state root for the L2
2. A list of txs to be scheduled by the L1
3. A list of messages (arbitrary outputs such as logs and events of smart contracts that are not transactions and are not routed)

Once validated, these three things together are the proof that the L2 transactions in the block were executed.

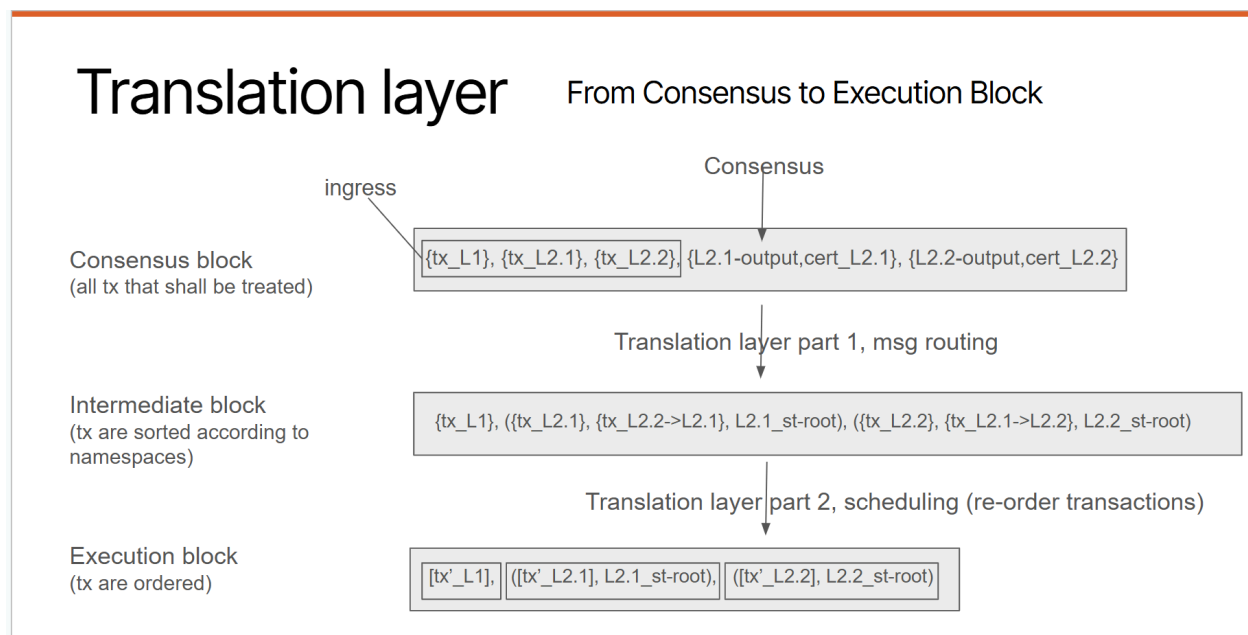
The "L2 block" might be executed later than the L1 block that the L2 block was contained in (the L2 could progress slower than the L1)

To accept an L2 block as executed, the set of validators executing the L2 each send their L2 block execution proofs in an L1 transaction targeting a precompile residing on the L1. This precompile accumulates the proofs and advances the L2 state once the validation criteria are met (e.g., in the threshold case 7 out of 10 executing validators have produced matching proofs, or in the ZK case a single one has produced a valid proof).

Together with recording the state advancement, the precompile is also responsible for scheduling the respective L1 transactions and routing transactions to their target L2s.

As future optimizations, note that this approach is wasteful on two fronts:

1. It requires waiting for the L1 to execute the proof-bearing transactions submitted by the subset of executing validators (which happens a few blocks later than their inclusion by consensus due to asynchronous execution). We can mitigate this if we settle on a fixed set of validity predicates (i.e., only threshold and ZK) -> as soon as the required proofs appear in consensus blocks, the L2 state can be considered advanced.
2. If the number of executing validators of an L2 is large, then the number of proof-bearing transactions submitted by these validators will be large, taking up block bandwidth. We can implement a separate sidecar protocol where these validators talk directly to compute an “L2 advancement” certificate that is included on-chain only once. Maybe using Victor’s new “Pool” primitive.



Certificates contain the list of ingested transactions that the output certificate is for.

Who pays gas? -> L2 validators need to be paid for that, so needs to be included into the gas fees for transactions.

Handling failure

High-level design for enshrined tokens (bridging)

From Tempo: they have “**bouncing back**” semantics: if a payment L2->L1 is rejected or fails for some reason, the money is sent back to the L2.

Also from Tempo: “composable withdrawals.”

ZKvm design consideration

The validator generates a proof that old-state + block -> new-state + messages.

To verify the proof one needs to have the old state and the block in an authenticated manner.

While the old-state can be authenticated with a proof that it was correctly derived from the prior state, the block is sequenced by the L1 and hence can only be authenticated by an L1 execution certificate or by having computed the L1 state itself.

We have two design options:

1. Gas payment authenticated by ZK-proof. If the proof can be fully verified, it can authenticate the gas payment of the L2. As the L2 can only have the block available once it is available on the L1 (possibly not certified), by the time the L2 validators submit the proof the block is available and the proof can be fully verified. Thus, the proof can be used to authenticate the gas payment of the messages the L2 validator submits.
2. Gas payment by prover and reimbursed later. If we do not want to rely on the block being available, or want to verify the proof only later, one can require the validator to have its own gas account from which gas is paid and then reimburse the gas once the proof is verified in the precompile.

Verification of ZK proofs is about 30 times more expensive than ECDSA verification. However, in the case of a ZKvm L2, only one proof is required to be verified, as the nonce will also be generated by the L2 and be included in the proof. Compared to say a threshold or multi

signature and assume one would want to run the computation replicated say 20 times and accept a threshold of signatures, one would have to verify 10 signatures (poor man's threshold) or a BLS signature, thus we have about a factor of two more compute for Groth16.

L2 to L2 messaging

- Bank A2 calls `bridgeToL1` function - takes in recipient address, token address, amount
 - Message inbox in the L2A
- When we prove state root of L2A in L1, all messages are in a commitment, posted on L1
 - Prove message existence against this commitment
 - Do whatever with the message

// Contract on L2B:

```
f.method1(stuff1, stuff2) {
```

```
}
```

```
f.method1'(Message m {stuff1, stuff2} )
```

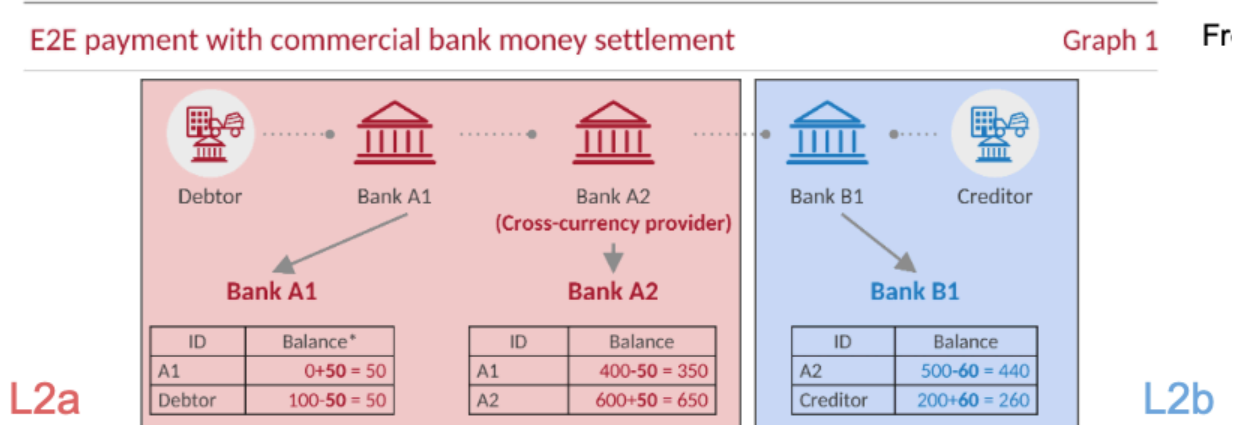
From

To

Data

Demo flow 1: User Ua on L2a wants to send money to User Ub on L2b

This flow corresponds to the setting



from the [BIS requirements document](#).

Setup:

There are two L2s: L2a and L2b.

On L2a there are two (wrapped) € ledgers, one controlled by Bank A1 and one by Bank A2:

- User Ua has an account on Ledger A1.
- Bank A1 has an account on Ledger A2.

On L2b there is one (wrapped) \$ ledger controlled by Bank B1:

- User Ub has an account on Ledger B1.
- Bank A2 has an account on Ledger B1.

Additionally, to coordinate the flow, there are two lock contracts:

- Lock A on L2a (with an account on Ledgers A1 and A2);
- Lock B on L2b (with an account on Ledger B1);

And a lock coordinator contract on the L1.

Bank A2 offers currency conversion as a service which is possible because it has an \$ account with B1 and runs a € ledger on L2a.

Demo:

User Ua (Debtor) wants to send 50€ to User Ub (Creditor), who has to receive 60\$. Ua has an account on bank A1 in €, while Ub has an account on bank B1 in \$. There is a bank A2 that has accounts in both currencies and will facilitate the currency exchange and offers FX as a service.

User A sends a setup message to lock_coordinator on the L1, specifying

- Lock A on L2a;
- Lock B on L2b;
- Timeout time t;
- context message (e.g., hash committing to details such as 50€ from Ua to 60\$ to Ub)

and gets a unique coordination id lock_cord_id. (The lock coordinator also emits an event tying the context message with the lock_cord_id).

The timeout time t defines when a party can request for the transfer to abort if not all involved locks are ready until that time.

Lock A on L2a (has accounts on ledgers A1 and A2)

- User Ua sends setup message
 - (lock_cord_id, context message)
 - Ua to lock 50€ on Ledger A1 for Bank A1 (by transferring them to lock A on Ledger A1)
 - A1 to lock 50€ on Ledger A2 for Bank A2 (by transferring them to lock A on Ledger A2)
- Ua sends 50€ on Ledger A1 to lock A.
- A1 sends 50€ on Ledger A2 to lock A.
- Once Ua and A1 have sent the money, lock A sends (lock_cord_id, "ready") to lock_coordinator on L1.
- Upon receiving (lock_cord_id, "fire") from L1 lock_coordinator:
 - Lock sends 50€ on Ledger A1 to Bank A1.
 - Lock sends 50€ on Ledger A2 to Bank A2.
- Upon receiving (lock_cord_id, "abort") from L1 lock_coordinator:
 - Lock sends 50€ on Ledger A1 to User Ua
 - Lock sends 50€ on Ledger A2 to Bank A1

Lock B on L2b (has account on ledger B1)

- User Ua sends setup message
 - (lock_cord_id, context message)
 - A2 to lock 60\$ on Ledger B1 for User Ub
- A2 sends 60\$ on Ledger B1 to lock B
- Once A2 has sent the money, lock sends (lock_cord_id, "ready") to lock coordinator on L1
- Upon receiving (lock_cord_id, "fire") from L1 lock_coordinator
 - Lock sends 60\$ on Ledger B1 to User Ub
- Upon receiving (lock_cord_id, "abort") from L1 lock_coordinator
 - Lock sends 60\$ on Ledger B1 to Bank A2

Lock_coordinator on L1

- Upon receiving ("setup", {Lock A, Lock B}, timeout, context message) from any party
 - Generate fresh lock_cord_id
 - Store ("Lock", lock_cord_id, {Lock A, Lock B}, timeout, context_message)
 - Emit ("Lock", lock_cord_id, {Lock A, Lock B}, timeout, context_message)
- Upon receiving (lock_cord_id, "ready") from Lock P
 - Check if ("Lock", lock_cord_id, Locks, ctxt) exist for some values of Locks, ctxt
 - Check if Lock P is in Locks
 - Store ("Ready", lock_cord_id, Lock P)
- Upon receiving (lock_cord_id, "fire") from any party
 - Check if ("Lock", lock_cord_id, Locks, timeout, ctxt) exist for some values of Locks, timeout, ctxt.
 - Check that for all P in Locks, there is a record ("Ready", lock_cord_id, Lock P).
 - Check that there is no record ("aborted", lock_cord_id)
 - Store record ("fired", lock_cord_id)
 - Send messages (lock_cord_id, "fire") to all P in Locks
 - Emit (lock_cord_id, "fired")
- Upon receiving (lock_cord_id, "abort") from any party
 - Check if ("Lock", lock_cord_id, Locks, ctxt) exist for some Locks, timeout, ctxt.
 - Check that there is no record ("fired", lock_cord_id)
 - Check that time > timeout
 - Store record ("aborted", lock_cord_id)
 - Send messages (lock_cord_id, "abort") to all P in Locks
 - Emit (lock_cord_id, "aborted")

Progress status:

- A1, Ua and Ub shall be able to view progress, i.e., they can ask A1, A2, B1, Lock A, Lock B, on the status whether they have locked the funds using their identity and the context message as access control to query the status. The state of the L1 Lock_coordinator is public and thus anyone can see the status there.
- A1, Ua and Ub shall be able to re-send the message to trigger the flows to A1, A2, B1, Lock A and Lock B in case a message failed because of low gas or because it was out of sync.

Privacy and compliance parts:

- Banks have an interface to check whether a user can receive funds from another user via a given path
 - (Ua -> A1 -> A2)_L2a -> (B1 -> Ub)_L2b
 - It will check whether L2b accepts transfers from L2a, B1 accepts A2, and whether there are no AML issues with Ua, Ub

- All information on the L1 is encrypted, it is only visible that Lock A and Lock B are involved in a transfer.
- Context message contains the necessary information encrypted for L2a and L2b.

Demo flow 2: User A on L2a wants to send money to User B on L2b using L1 liquidity to swap the tokens User A has to the token User B wants.

- User A has an € account with Bank A on L2a
- User B has an \$ account with Bank B on L2b
- User A wants to send 60\$ to User B using his € account
- All tokens are rooted in the L1
- Bank A2 now operates liquidity pool on the L1
- The liquidity pool on the L1 has a locking function (call it lock LP). This is the only lock we use. It locks tokens at the current exchange rate (at a cost, which we ignore for the demo)
- Bank A can charge user A's account with the correct amount. When receiving the input token, it will send the output token to a specified address.
- Bank A will ask Bank B whether user B is KYD for 60\$. If the reply is positive, it asks the LP to lock the 60\$ and will get back an answer that this costs 50\$.
- The scenario includes a KYC inquiry: Bank A sends a message to Bank B to inquire whether the intended recipient exists and is allowed to take the funds.

Setup:

There are two L2s: L2a and L2b.

On L1 there are two ledgers, one for € and one for \$.

- There is a liquidity pool that has accounts on both ledgers and offers a locking mechanism for securing swaps at a given exchange rate.
- L2a has an account on the € ledger.
- L2b has an account on the \$ ledger.

On L2a there is a (wrapped) € ledger controlled by Bank A.

- User Ua has an account on Ledger A.

On L2b there is one (wrapped) \$ ledger controlled by Bank B:

- User Ub has an account on Ledger B.

Flow:

1. User A sends a message to Bank A requesting to send 60\$ to User B at Bank B. As part of that message, the user authorizes Bank A to charge the user's account with the appropriate amount of €.

2. When receiving the request, Bank A sends Bank B a message to inquire whether User B at Bank B can receive the 60\$ from User A.
3. Obtaining a positive answer from Bank B, Bank A requests a liquidity lock at LP on L1 for 60\$ to be sent to user B at Bank B on L2b. As a reply it will receive a quote of 50€ and a lock_id. The LP lock will have a time out.
4. Bank A takes 50€ out of user A's account and sends it to LP's account on the L1 with the lock_id as the memo..
5. Upon receiving 50€ from L2a with the lock_id as memo, the LP will send the 60\$ to User B at L2b.
6. In case the transaction of 50€ to the LP happens after the LP lock has timed out, the LP is set up to return 50€ (possibly minus a fee) to the encrypted address user A at Bank A on L2a.

References

<https://tempo.xyz/blog/deel-stablecoin-wallet-global-contractors-tempo/>

Spoke/Hub interface:

```
sendMessage(dest_namespace : ChainIdType, to : Address, data :  
CallldataType)
```

Imagine a call from **caller** to Spoke.sendMessage(n, t, c) on namespace **X** (!= L1):
This emits a message:

```
m_encr = EncrMsg {  
  src_namespace = X  
  dest_namespace = n  
  encr_payload = encr(key(n), {  
    From = caller  
    To = t  
    Data = c  
    nonce = ...  
  })  
}
```

m_encr gets committed to the state of Spoke on **X**.

After L2 execution for a block is done, the L2 do the following:

1. Get the new state root s_root;
2. Given all messages sent in this block: [m_encr1, m_encr2, m_encr3, ...] get a merkle state root messages_anchor.
3. Take the State root and execute an L1 transaction calling hub.advance(newStateRoot, messagesAnchor)
4. This may take multiple L2 validators doing this until the hub considers message m_encr committed: i.e., the corresponding state root landed.

Afterwards, the L2 validators may begin delivering messages, including m_encr. To deliver m_encr, they create an L1 transaction calling hub as hub.deliverMsg(m_encr, proof).

...

Constraints:

- Every hub/spoke has mechanism "committed(msg, evidence): bool" -> this only returns true upon being convinced that the given message was emitted by the sending namespace of the respective message, full stop. Liveness: eventually the proof accepts.

L2A -> L2B flow:

L2A: In a tx: Spoke.sendMessage("L2B", to, data)

The resulting artefact is **encr_msg m1**.

m1 is stored in the spoke state.

(note, m1 needs to be deterministic)

Then on L1:

Someone calls Hub.dispatch(m1) in a tx.

The hub then does the following checks:

- if (m1.dest_namespace != "L1") { // We assume we take the if branch, see the other flow for not.
- Emit "

// Actually does the L1 need to dispatch messages that are not intended for it? It's clear what should happen on the receiving L2: there we properly dispatch.

Let's do two methods:

- Hub.dispatch(msg) -> asserts dest == L1
- Hub.relay(msg) -> verifies it was sent on the sending namespace, if so, places the same message in the outbound queue of the L1 (this will be included in the anchors the corresponding L2 sees).

Proving flows (completely separate)!

Plaintext Messages

A plaintext message is represented as follows:

```
p_msg = PlainMsg {  
    source_n, dest_n // Source and destination namespaces  
    seq_no,          // Sequence number for source_n, increments at each emitted msg  
    from,            // Caller in source_n  
    to,              // Receiving contract in dest_n  
    body             // Opaque blob  
}
```

Processing Messages (what receiving contracts program for)

Contracts that want to process an incoming messages MUST implement a method:

```
C.processMessage(PlainMsg p_msg)
```

Only plaintext messages may be processed.

The caller will always be H/S. The contract can assume that `p_msg.to == this`, and that deduplication has already been done (they will not receive the same message for processing twice). Contracts MUST require(`caller == H/S`) to ensure the `processMessage` endpoint achieves its guarantees. Monad MAY choose to enforce that only the H/S may call methods named “`processMessage`”. In case Monad does not enforce this and a contract does get called with a bogus Message, it’s on the contract. Confidentiality of messaging is always ensured (cf. next sections): proper correct messages only get decrypted and passed to the correct recipient by the protocol.

Encrypted Messages

An encrypted message is represented as follows:

```
c_msg = EncrMsg {  
    source_n, dest_n,  
    seq_no,  
    salt,  
    from = PlaintextFrom(p_from) | CiphertextFrom(c_from),  
    to = PlaintextTo(p_to) | CiphertextTo(c_to),  
    body = PlaintextBody(p_body) | CiphertextBody(c_body)  
    ... -> add symmetric key in the future.  
}
```

What fields will be encrypted depends on the source and destination namespaces, and a message with other combinations of encrypted fields is considered invalid.

source_n	dest_n	from	to	body
L2	L1	plain	plain	plain
L2	L2	ciphertext	ciphertext	ciphertext
L1	L2	plain	both allowed	both allowed

Sending Messages

On every namespace there is a contract, called the Hub on the L1 and the Spoke on any L2. We will collectively refer to the two as the namespace Hub/Spoke (H/S).

The H/S of a namespace exposes a method:

`H/S.sendMsg(dest_n, to = PlainTo | PreEncrTo, body = PlainBody | PreEncrBody)`

The caller can choose to provide some of “to” and “body” already encrypted under the destination namespace’s public key. After `sendMsg` runs, the output is an `EncrMsg`. Fields that were already provided encrypted stay encrypted, while fields that have to be encrypted but were provided in plaintext get encrypted. If a field that must be in plaintext is provided encrypted the function reverts.

Semantics:

```

sendMsg(dest_n, to, body): [Initialize: current_seq_no = 0]
- from = caller
- salt = PRF(current chain_id_seed, "salt" || current_seq_no)
- r    = PRF(current chain_id_seed, "enc" || current_seq_no)
- Emit EncrMsg {
    source_n = current chain_id,
    dest_n = dest_n,
    seq_no = current_seq_no,
    salt = salt,
    (from, to, body) = PromoteEncrypt_{pk_{dest_n}}(from, to, body; r)
  }
- current_seq_no += 1

```

Example: say on chain_id X, caller C makes a call `H/S.sendMsg(Y, to, body)`, the outcome is a message:

`c_msg = EncrMsg {`

```

    source_n = X, dest_n = Y,
    seq_no = <assigned at call-time in ascending order>,
    salt    = PRF(X_seed, "salt" || seq_no)
    (from, to, body) = PromoteEncrypt_{pkY}(C, to, body; PRF(X_seed, "enc" || seq_no))
}

```

This is encrypted for the receiving namespace, meaning that it is to be consumed by the respective namespace and no other, and only by “to”.

The creation of this message gives a guarantee of an eventual right to decrypt and fire it in the destination namespace.

Dispatching Messages (decryption, authenticity and replay-protection)

The H/S have a method `.convinced_of_existence(c_msg, evidence)` with the following properties:

- If the message is not something that was actually sent, it returns false.
- If the message has actually been sent, it can return true or false.
- The guarantee is that eventually, valid evidence to prove the message will exist.

See later for how these guarantees are achieved.

To decrypt and dispatch an encrypted message to the receiving “to” on the receiving namespace, the H/S provide the following interface:

```
H/S.dispatchMsg(EncrMsg c_msg, Evidence ev)
```

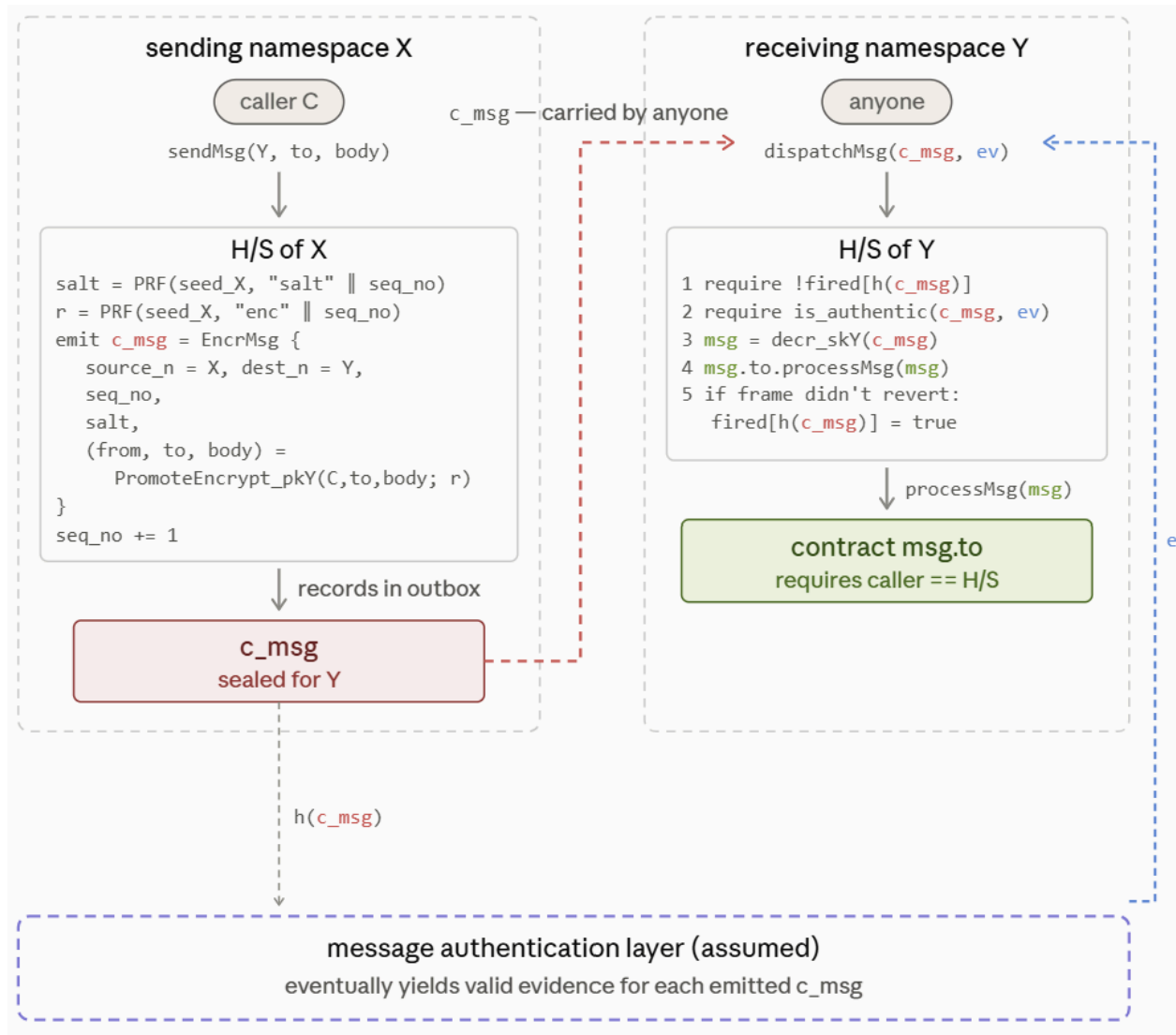
Calling it requires supplying evidence that said message exists and was indeed sent as claimed. The format and mechanism for gathering such evidence is explained later.

Semantics:

```

H/S.dispatchMsg(c_msg, ev): [Initialize once: set fired = empty]
- asserts c_msg.dest_n == current chain_id.
- assert not fired[h(c_msg)]
- assert convinced_of_existence(c_msg, ev) // Returns “yes” or “not (yet)”.
- msg = decr_{sk_{current chain_id}}(c_msg) // Decryption.
- Call msg.to with calldata = “processMessage(msg)”. // Dispatch call.
- If the dispatched frame didn’t revert: fired[h(enc_msg)] = true

```



Merkle Commitments

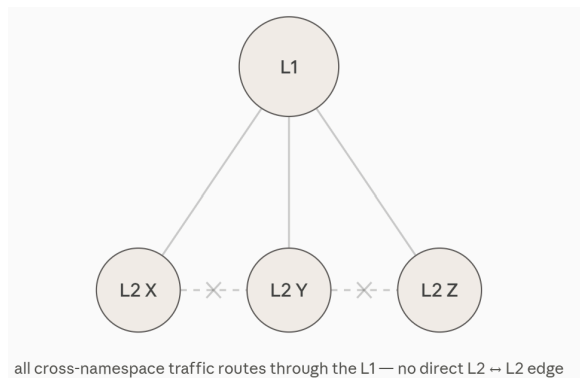
An encrypted message `c_msg = EncrMsg { source_n, dest_n, seq_no, salt, from, to, body }` is hashed as `h(c_msg) = keccak("encr_msg" || source_n || dest_n || seq_no || salt || from || to || body)`. For reasons to become apparent in a second, we refer to this hash as `c_msg`'s *leaf hash*. Given a *batch* of encrypted messages `B = (c_msg[1], ..., c_msg[k])`, we define `B`'s *messages anchor* to be the root hash of a Markle tree with leaves `h(c_msg[1]), ..., h(c_msg[k])`:

`ma(B) = MerkleRootHash(h(c_msg[1]), ..., h(c_msg[k]))`

One can prove that a given message `c_msg` belongs to a batch `B` by supplying `ma(B)` as well as a Merkle path from the leaf `h(c_msg)` to the root `ma(B)`.

Cross-Namespace message transport and authentication

Message transport logically happens on a star topology with the L1 in the center and virtual communication links joining the center with all L2s.



In particular, an $X \rightarrow Y$ message does not get to `Y` directly, but first needs to settle on the `L1` as an intermediary.

Concretely, the virtual communication links materialize through message outboxes and anchor inboxes.

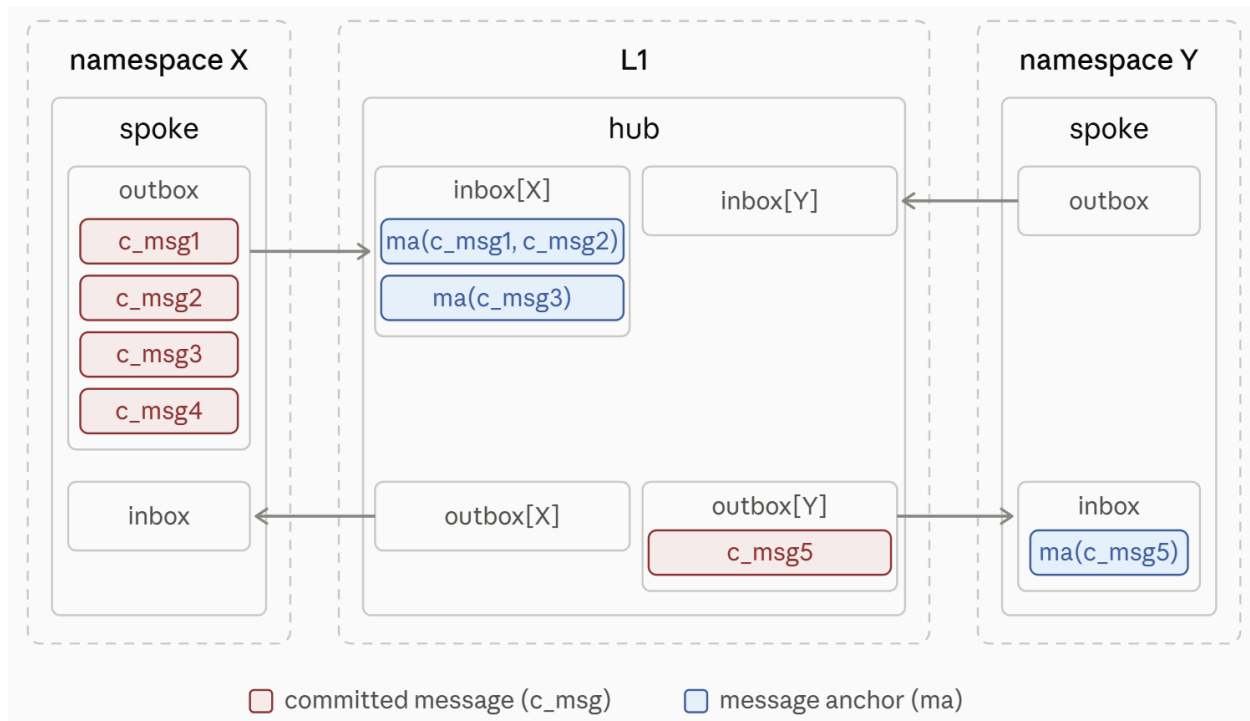
Message Outboxes:

- Each spoke has a `Spoke.outbox` storing all messages emitted by the spoke.
- The hub has outboxes keyed by other namespaces' chain ids `Hub.outbox[chain_id]` storing all messages emitted by the hub with `dest_n = chain_id`. This includes messages emitted by a different namespace and re-emitted/forwarded by the `L1`.

Anchor inboxes:

- Each spoke has a `Spoke.inbox` storing message anchors for batches of messages emitted by the hub.
- The hub has inboxes keyed by other namespace's chain ids `Hub.inbox[chain_id]` storing message anchors for batches of messages emitted by the spoke of `chain_id`.

In particular, every outbox is keyed by the id of the namespace meant to consume the messages, and corresponds to an inbox on the respective namespace.



The interface of the hub and spokes to record new message anchors into their inboxes:

S.recordAnchor(msgsAnchor)

recordedAnchors[msgsAnchor] = true; // Initially all false.

H.recordAnchor(chain_id, msgsAnchor)

recordedAnchors[chain_id][msgsAnchor] = true; // Initially all false.

Crucially, these functions will not be callable “at will”, i.e., with incorrect anchors, but rather will be called only in contexts where the authenticity of the messages anchor can be ensured. Details come later.

Armed as such, we can now explain how “authenticate” works. Fix the shape of the evidence we provide for a message `c_msg` as:

```
Evidence {
  anchor,    // a Merkle root hash of a message batch containing c_msg
  path      // Merkle path from the leaf h(c_msg) to the anchor.
}
```

Then the message authentication methods on the hub and spokes work as follows:

```

H.is_authentic(c_msg, Evidence ev)
  return recordedAnchors[c_msg.source_n][ev.anchor]
    && MerkleProof.verify(ev.path, ev.anchor, h(c_msg));

```

```

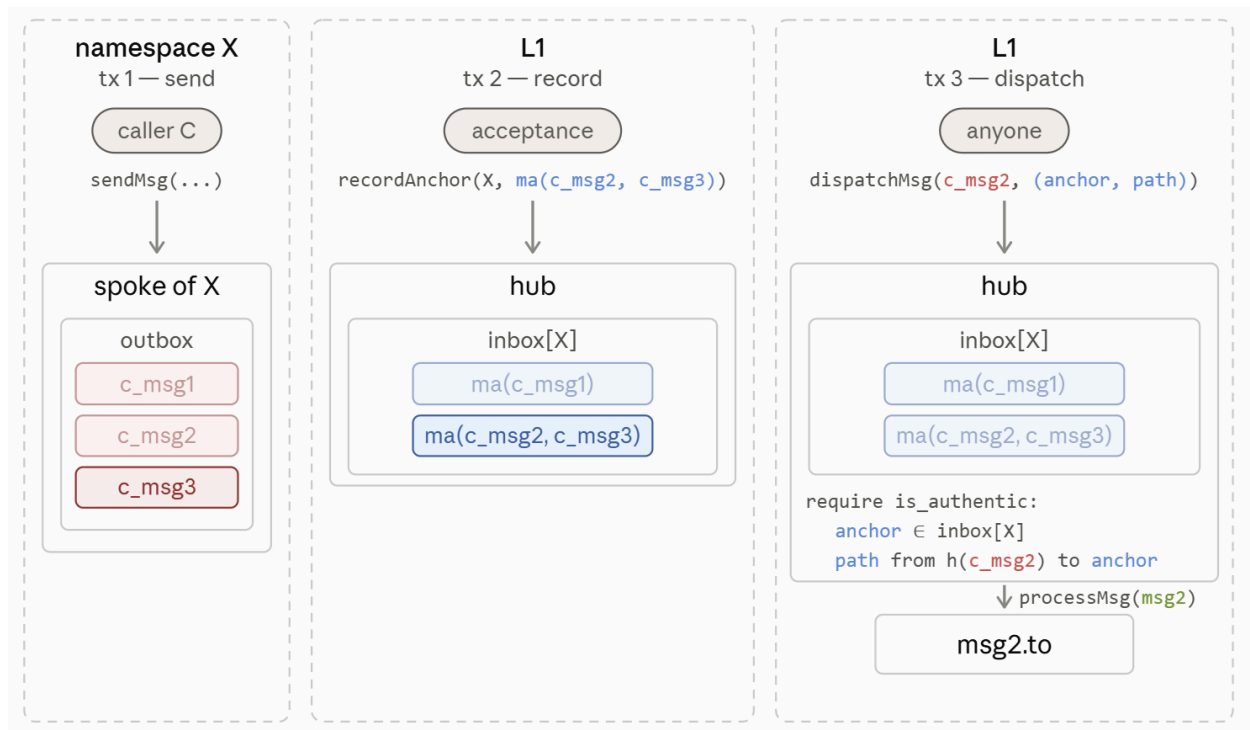
S.is_authentic(c_msg, Evidence ev);
  return recordedAnchors[ev.anchor]
    && MerkleProof.verify(ev.path, ev.anchor, h(c_msg));

```

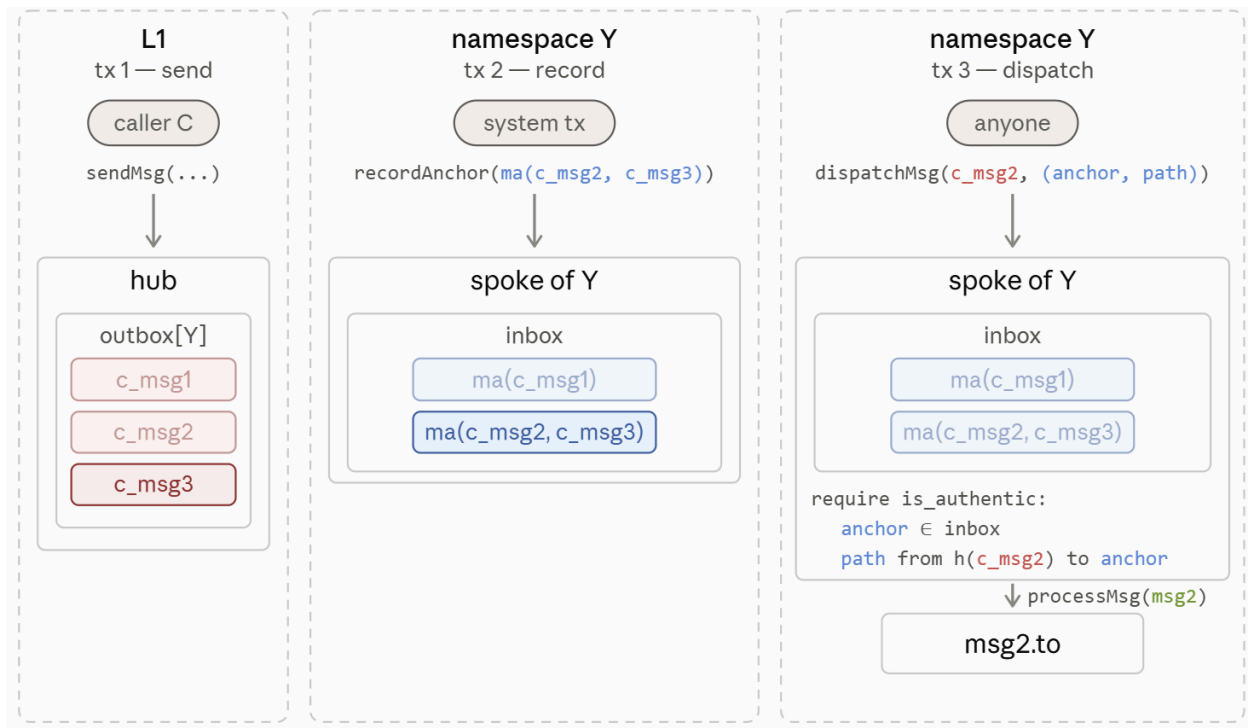
In particular, to authenticate a message (the prerequisite to dispatching it), one has to provide a message anchor that landed in the destination namespace and a proof of inclusion of the claimed message within.

The following diagrams show the full life cycle of an L2→L1 and L1→L2 message.

L2→L1:



L1→L2:

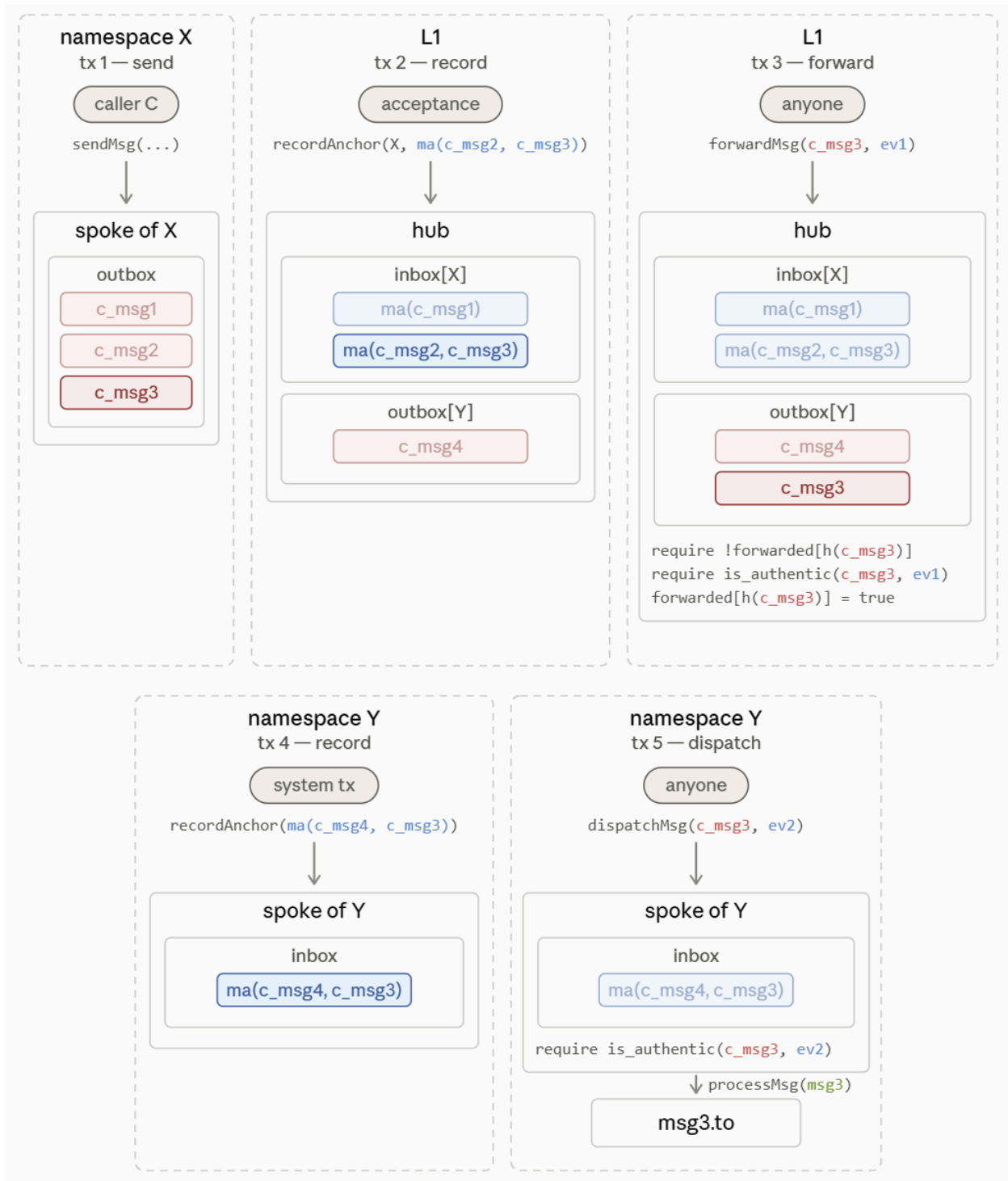


Forwarding L2→L2 Messages (will be a subsection)

The previous explained the two simpler cases: L2→L1 and L1→L2 messages, i.e., those where the message can directly land on the destination namespace. L2→L2 messages require forwarding through the L1 to respect our star topology. We achieve this by adding another method to the hub:

```
H.forwardMsg(c_msg, ev): [Initialize once: set forwarded = empty]
    assert c_msg.dest_n != L1
    assert not forwarded[h(c_msg)]
    assert is_authentic(c_msg, ev)
    emit c_msg // Re-emits the message in the L1's outbox.
    forwarded [h(c_msg)] = true
```

As a result, an L2→L2 message requires a 5-transaction flow, as follows:



In particular, for a message to become fireable on the destination L2, it first needs to be forwarded/added to the L1's outbox for the destination L2, which first requires a corresponding anchor to land in the L1's inbox for the source L2. This means that the L1's outboxes will contain both L1→L2 messages, as well as L2→L2 messages it re-emits.

L2 transaction format and execution semantics

An Ethereum transaction (EIP-7702) has the following format:

```
tx = Tx {  
  chain_id,  
  nonce,           // sequence number for the sender  
  max_priority_fee_per_gas, max_fee_per_gas,      // gas price bids  
  gas_limit,       // max gas the tx may consume  
  to,              // recipient  
  value,           // native amount transferred  
  data,            // calldata  
  access_list, authorization_list,               // irrelevant here  
  y_parity, r, s   // sender's signature, "from" derived from it  
}
```

Where the chain_id binds the transaction to a chain: chain_id = 1 for ETH main-net and = 143 for Monad mainnet. Transactions for a given Monad L2 namespace will use the respective chain_id. It is the responsibility of the namespace operator to choose a chain-id that isn't widely used by any established public project.

L2 namespace transactions will respect the following additional constraints:

- value = 0 (there exists no native value on the namespaces, native value can be bridged through WMON)

and will run with modified semantics:

- The two gas price bid fields max_priority_fee_per_gas and max_fee_per_gas are ignored (Andrei: I would go as far as requiring them to be 0).
- All gas payments are skipped, the fees for running the transactions are assumed to be prepaid at sequencing time on the L1 (see later).

L2 blocks look like normal execution blocks, with the difference that invalid transactions do not automatically invalidate the block: the transaction is just skipped. This is because, at sequencing time, certain L2 transaction fields are provided encrypted (see next section), meaning that L2 transactions cannot be fully validated at sequencing time as L1 transactions would.

L2 encrypted transaction submission and gas payment on the L1

When a user wants to sequence an L2 transaction **tx** for an L2 namespace, they first encrypt the transaction into the following format:

```

c_tx = EncrTx {
  chain_id,      // of the respective L2 namespace
  gas_limit,
  maybe = NORMAL_TX vs | MSG_TX
  encr_envelope = Encrypt_{pk_chain_id}({
    nonce,
    max_priority_fee_per_gas, max_fee_per_gas,
    to,
    value,
    data = "dispatchMsg(...c_msg6...)",
    access_list, authorization_list,
    y_parity, r, s
  })
}

```

For the purposes of this document, we assume that L2 gas has a fixed constant price `L2_GAS_PRICE`. Future extensions will let this be configurable or adapt to demand of each specific L2. L2 gas costs are debited at sequencing time on the L1 and moved to an operator-designated address recorded in the L1 hub, `Hub.gasAccount[chain_id]`.

To request sequencing `tx`, the user creates an L1 transaction `tx_L1` calling a designated hub method with `c_tx` in the call data and supplying money to cover for `tx`'s L2 gas cost in `tx_L1`'s value field. In particular, the hub method and associated semantics are:

```

Hub.sequenceNamespaceTx(EncrTx c_tx): [payable, gets value from caller]
  fee = L2_GAS_PRICE * c_tx.gas_limit
  assert value >= fee
  transfer fee to gasAccount[c_tx.chain_id]    // the namespace's gas reserve account
  transfer value minus fee back to caller        // refund the excess
  emit c_tx                                    // record c_tx as accepted for sequencing

```

It is particularly illustrative to consider an example to see how the L1 transaction wrapping the L2 encrypted transaction looks:

Alice wants to submit an L2 transaction `tx` for namespace `X`. She encrypts the required fields with respect to `X`'s public key to get a corresponding encrypted transaction `c_tx`. Then, she wraps `c_tx` into a normal L1 transaction `tx_L1` targeting the L1 hub for sequencing:

```

tx_L1 = {
  chain_id = L1,
  gas_limit = <required to run the L1 hub code>,
  value = (at least) tx.gas_limit x L2_GAS_PRICE,

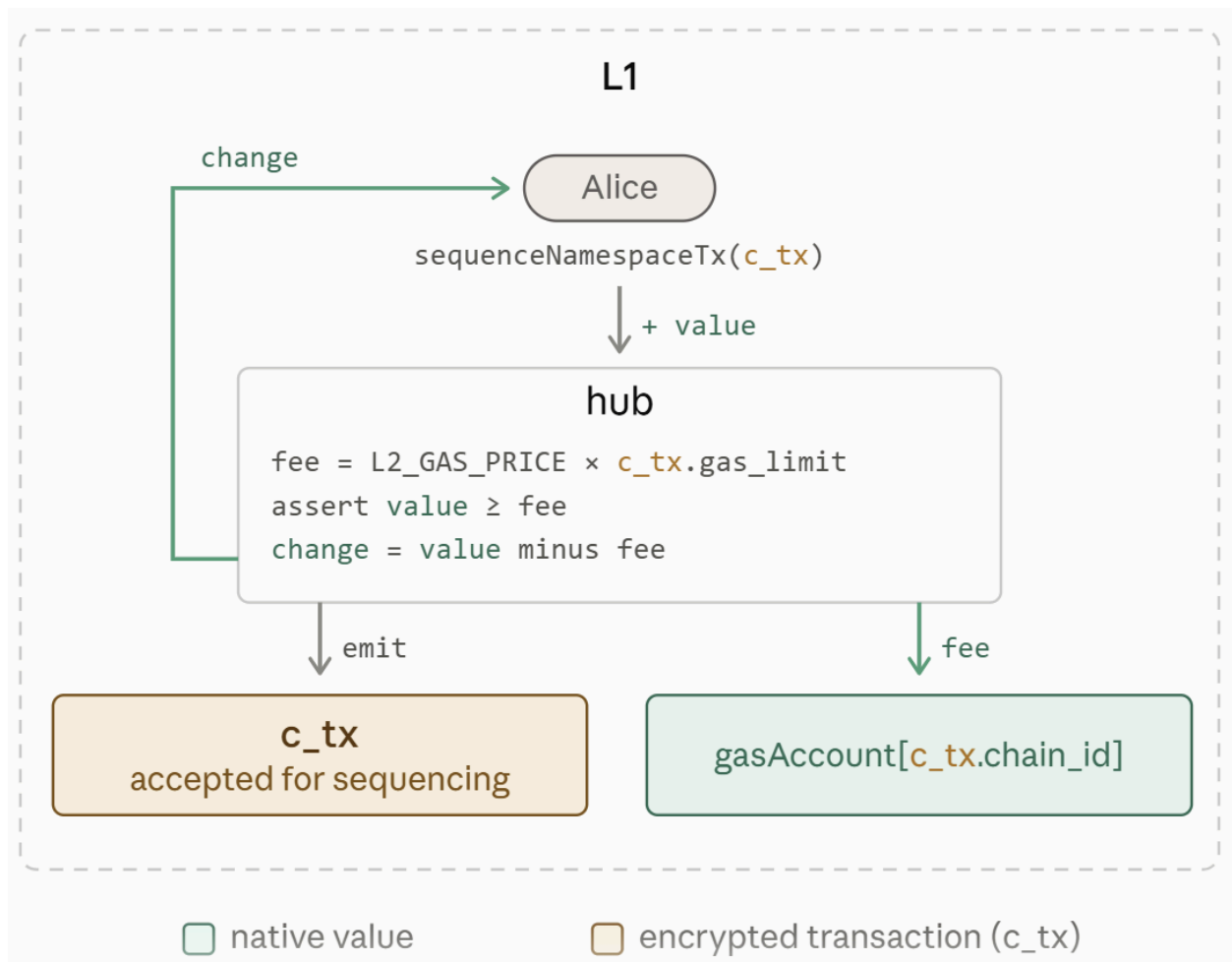
```

```

nonce = <current sequence number for Alice's L1 account>,
to = Hub,
data = "sequenceNamespaceTx(c_tx)",
y_parity, r, s // Alice signature on tx_L1
}

```

where internally `c_tx` designates a transaction with its own `gas_limit`, which is paid for through `tx_L1.value`.



Remarks:

- **L1 gas vs L2 gas:** `tx_L1` pays for its own L1 gas fees through the normal L1 mechanism, and covers for the gas costs of the L2 transaction `c_tx` through the additional money transferred in the `value` field (as a normal EVM value transfer).
- **What happens to the L2 gas money afterwards?** The L2 gas money is transferred into the operator-designated account `Hub.gasAccount[chain_id]`. We intentionally do not prescribe a single mechanism for distributing this money, e.g., to reimburse the L2

validators. This can happen out-of-band at the operator's discretion. In the real world, the operator would have SLA contracts with said validators, promising periodic out-of-band reimbursements in exchange for the provided service of running the namespace infrastructure.

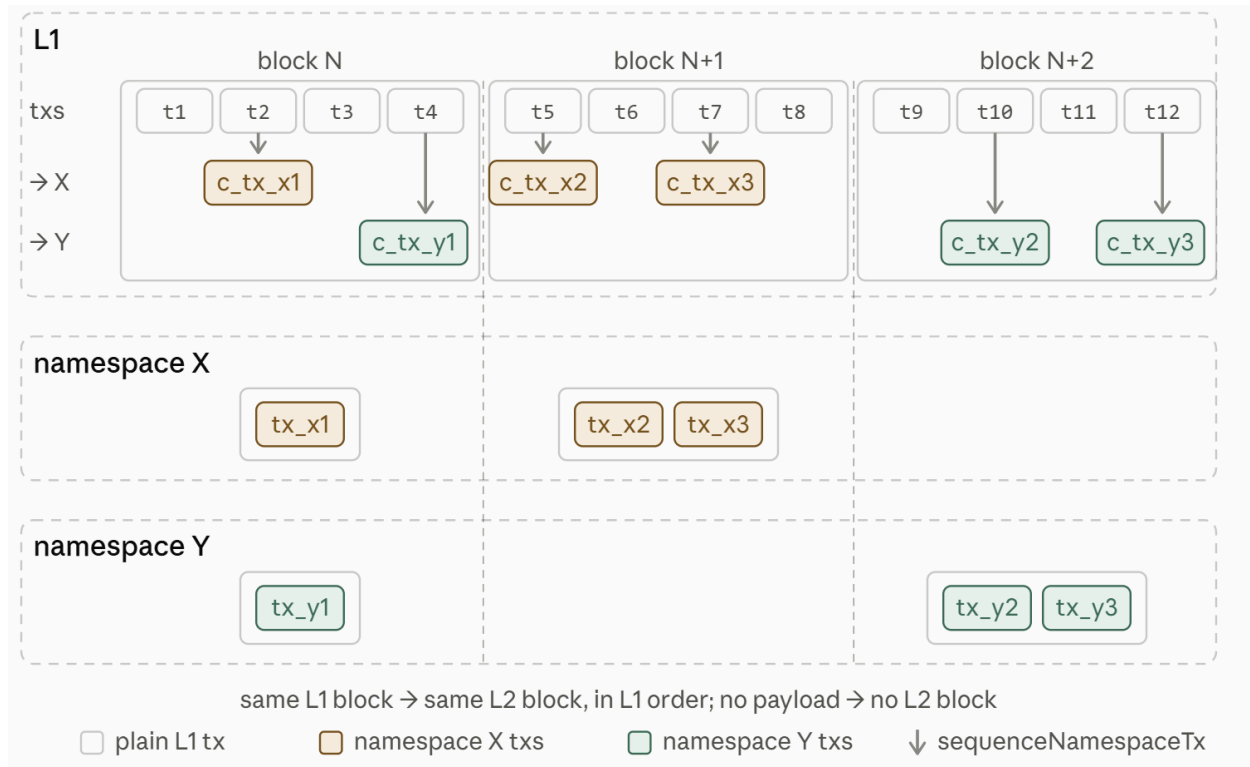
- **Gas sponsorship:** A by-product of the design is that it enables seamless gas sponsorship: Alice's L1 transaction can actually call the hub with `c_tx` being Bob's transaction. The internal `c_tx` is signed by Bob, but its gas is sponsored by Alice, as she sends the enclosing L1 transaction. This is also why refunds go to the caller rather than the inner transaction's signer: Alice paid, Alice gets the change.
- **Decrypting transactions:** it is to be understood that L2 validators decrypt the namespace transactions before executing them (under the namespace's secret key). This will be made explicit later when we discuss forming and decrypting L2 blocks.
- **FAQ:** the reader should be mindful of the distinction between encrypted transactions and encrypted messages. Transactions represent one atomic step executed in a namespace, while messages are logical objects facilitating inter-namespace communication. Both get their own encryption. For instance, Alice might submit an L2 transaction `c_tx` whose calldata may or may not involve dispatching or forwarding a message. If Alice's transaction involves decrypting a message, this is done at the protocol smart contract layer as specified earlier. Transaction encryption and decryption, on the other hand, are part of the L2 block-building and execution flow.

L2 block formation

The previous explained how L2 transactions are accepted for sequencing: L2 transactions were emitted by the L1 hub, i.e., accepted for sequencing, but not yet included or ordered into definite L2 blocks. We will need a mechanism for forming the L2 blocks.

We make the following two assumptions with respect to L2 block derivation:

- **FIFO sequencing.** For each L2 namespace, accepted transactions enter the L2 ledger in exactly the order of the L1 transactions that sequenced them. Concretely: if `tx_L1` precedes `tx_L1'` in the L1 ledger — earlier block, or earlier position within the same block — and they sequence `c_tx` and `c_tx'` for the same namespace, then `c_tx` precedes `c_tx'` in the respective L2 ledger.
- **Lock-step sequencing.** An L1 block that accepts at least one transaction for a namespace induces exactly one L2 block for it, containing precisely those transactions; an L1 block that accepts none induces no L2 block. In particular, two accepted transactions share an L2 block if and only if their sequencing transactions shared an L1 block.



L2 validators observe the L1 chain and pull in L2 transactions, decrypting them and forming the required L2 blocks.

Remarks:

- Later, when discussing how message anchors are accepted by the different namespaces, we will need to add an additional system transaction to each L2 block calling `acceptAnchors()` on the spoke (to ingest the respective message roots).... TBC

L2 execution result reporting and validation

Additionally, we will need a mechanism for L2 validators to report back and authenticate block execution results and message state anchors (allowing one to authenticate dispatching or forwarding messages). For this, we will need a couple of ingredients.

Later, the “backpressure rule”.

New requirement: proofs of L1 execution routed so the L2 execution sees quorum certificates for the L1. Hardest part:

- **Exposing consensus quorum certificates.**
- **Required so bank can verify the L1 is correct.**
- **Question: when I ask for all Monad blocks: how do I check it's the correct ones:**
 - **Do we persist consensus blocks (which have QCs)?**

For block building, we will assume a lock-step cadence:

Sequencing and L2 transaction ingestion

Format-wise L2 transactions are normal transactions with the `chain_id` set to the L2's

Namespace Execution Authentication Modes

“Attestation modes”:

- 1) One by one policy of multi-sig.
- 2) All-in-one multi-sig.
- 3) ZK
- 4) TEE...

Retries and Out-Of-Gas etc., backpressure.

Requirement: need to ensure backpressure in the L1.

Maybe: prioritising senders

L2s can signal to hub...

Discussing with L2 team.

```
L1 tx: {  
  l1_gas_limit = low  
  calldata = "sequence_L2_tx(... payload ..., l2_gas_limit = high)"  
}
```

[DRAFT]

Evidence/Proving Message Authenticity/The Message Commitment Layer/Message transport without L2->L2

Semantics: namespace “vouching_n” is vouching that all messages included in msgsAnchor are correct.

// This is only called under special circumstances:

- On the L1, it is only called together when an “advanceState(newroot, messages)” is done: advanceState takes as inputs basically the new state root, a proof for it...
- On the L2, it is called in a system transaction.
- Both of these things require going into batching and proving flows first.

// To be convinced of a message’s authenticity, on the L1, the sending L2 must have vouched for it, while on an L2, the L1 must have vouched for it.

```
H/S.convinced_is_committed(c_msg, Evidence ev)
    vouching_n = /* on the L1: c_msg.source_n
                  on an L2: L1 */;
    return recordedAnchors[vouching_n][ev.anchor]
        && MerkleProof.verify(ev.path, ev.anchor, h(c_msg));
```

To continue? Figure out (still at smart contract layer) how anchors are created:

- For now still simple: you can always say 1) there are outboxes 2) each block we compute a new anchorsRoot 3) this then “makes it” to acceptAnchor()
- For now good, but this eventually needs to be tied to advancing the state.

Every hub/spoke maintains a list of message anchors that it:

For now, let us explain the message commitment flows (how a H/S can be convinced of a message) without the L2->L2 case, which will come as an extension.

After executing each L2 block, the L2 validators take all messages added to the outbox in this last block: `c_msg1`, `c_msg2`, ..., `c_msgk`, hash them into *leaf hashes* `h(c_msg1)`, ..., `h(c_msgk)` and compute a Markle tree root hash over the leaf hashes `ma = Merkelize(h(c_msg1), ..., h(c_msgk))`.

The L1 hub has a method `H.accept`

Ok things to cover to even touch on all I need:

- Each channel A->B has an outbox on A and an anchors sequence the other way.
- Sending places the message in the outbox

The purpose of this section is to specify how a hub/spoke may be convinced of the authenticity of a message at dispatch time: message dispatch should always fail for a bad message. `H/S.dispatchMsg(c_msg, ev)` makes a call to `H/S.convinced_is_committed(c_msg, ev)`.

Specification:

The method `convinced_is_committed(c_msg, ev)`

Each namespace hub/spoke maintains a message outbox. For the L1, this is split into one lane per L2 `chain_id`.

Common hub/spoke (HS) interface: (message execution interface)

- `H/S.dispatchMsg(encr_msg, evidence)`
- `H/S.convincedOfAuthenticity(encr_msg, evidence)`, or `.requireCommitted()`

Then separately, the proving flow (this needs liveness), for which we have an extra L1 method `.relayMsg(e_msg, evidence)` -> unlike L1 dispatch, this only re forwards the message to the queue of the destination L2.

The outbound queues of hub contain both messages enqueued directly as well as

`relayMsg(encr_msg, Evidence):`

- Checks evidence it's convinced it exists. If yes, enqueue as if it sent the message in the first place to the receiving L2.

H/S.learnNewMessageAnchors(msgAnchors) (that ultimately gets called under different circumstances in the L1 and L2s) gets called either from the advancement function or in L2 blocks from system-injected txs.

On L1 it's only called from one method inside the hub itself when the state advances. On the L2, it can only be called by the system sender, once per block to learn of the new messages to be "learned" with each L2 block.

Notable flow we are not doing:

```
msg1 = {  
  to = L1.router  
  data = { ... instructions for what to do downstream <actual target> }  
}
```

The router happens to have logic to spawn a message

```
msg2 = {  
  from = L1.router  
  to = actual target  
}
```

WE NEED TO CONSIDER BYZANTINE SUBNETS FORGING.

Normally, zkEVM pes `state_root1 -> {Block} state_root2`

While here we prove:

- `state_root1 -> {Block} (state_root2, message_anchors_to_add)` // proof of this.

Proof:rov

- Input-block as appear on Hub ->{decryption key} commitment(transformed & decrypted block)

- State_root_1, commitment(transformed...) -> {decrypted block} (state_root_2, message_anchors_to_add)

Salted state root = h((state root, h(block_height, source of randomness))

c_a -> work L1 -> c_b
L2A — L1 — L2B

L2A:

c_a -> sendMsg(dest_namespace = L2B, to = c_b, ...):

- Produces an encrypted message under L2B's key, which has everything encrypted.
- The "from" is set to c_a.

The Kai flow:

L2A:

c_a -> sendMsg(---dest_namespace = L1— (not exist), to = L1.router, data)

```
data = {
  "Please create the a second message L1->L2B"
}
```